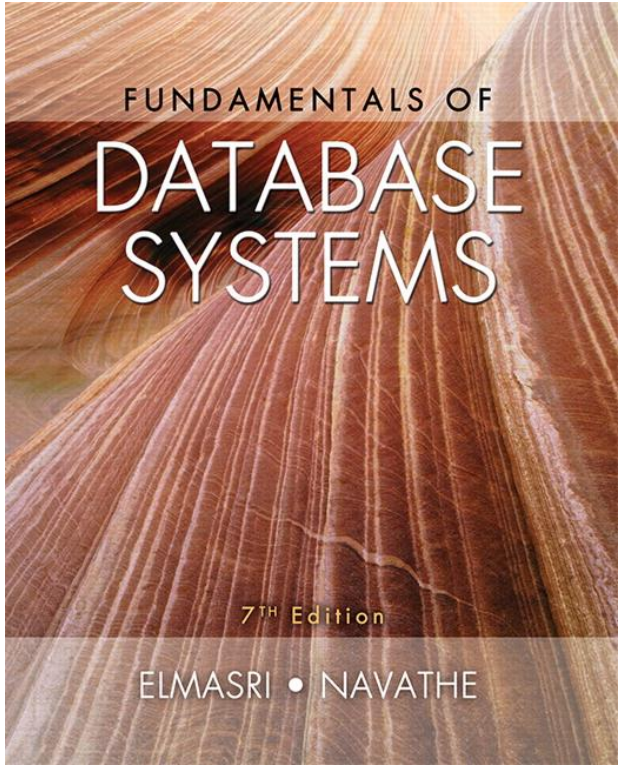


Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

CHAPTER 21

Concurrency Control Techniques

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Introduction

- Concurrency control protocols
  - Set of rules to guarantee serializability
- Two-phase locking protocols
  - Lock data items to prevent concurrent access
- Timestamp
  - Unique identifier for each transaction
- Validation or certification of a transaction

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

LAST CLASS

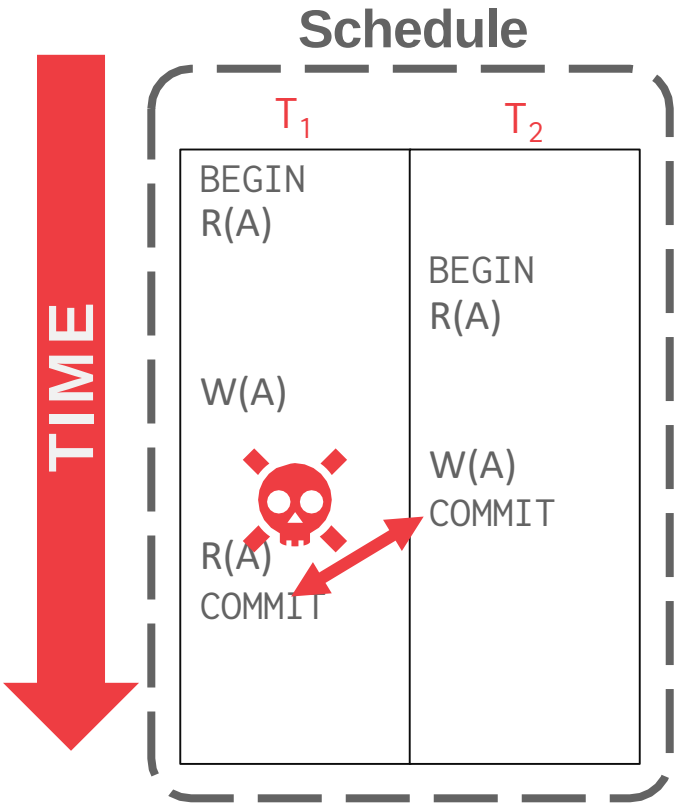
Conflict Serializable

→ Verify using either the "swapping" method or dependency graphs.

→ Any DBMS that says that they support "serializable" isolation does this.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

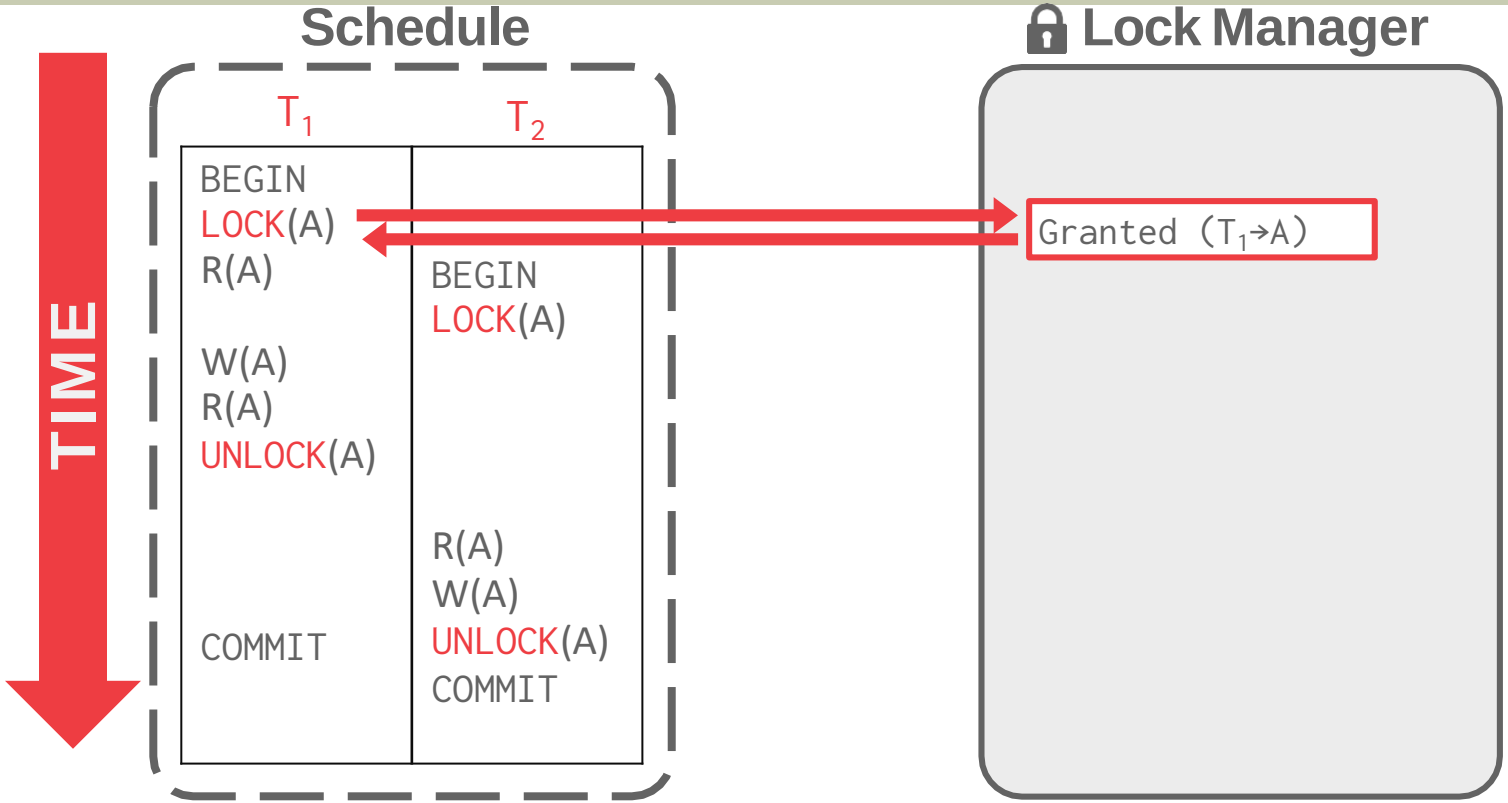
OBSERVATION

We need a way to guarantee that all execution schedules are correct (i.e., serializable) without knowing the entire schedule ahead of time.

Solution: Use **locks** to protect database objects.

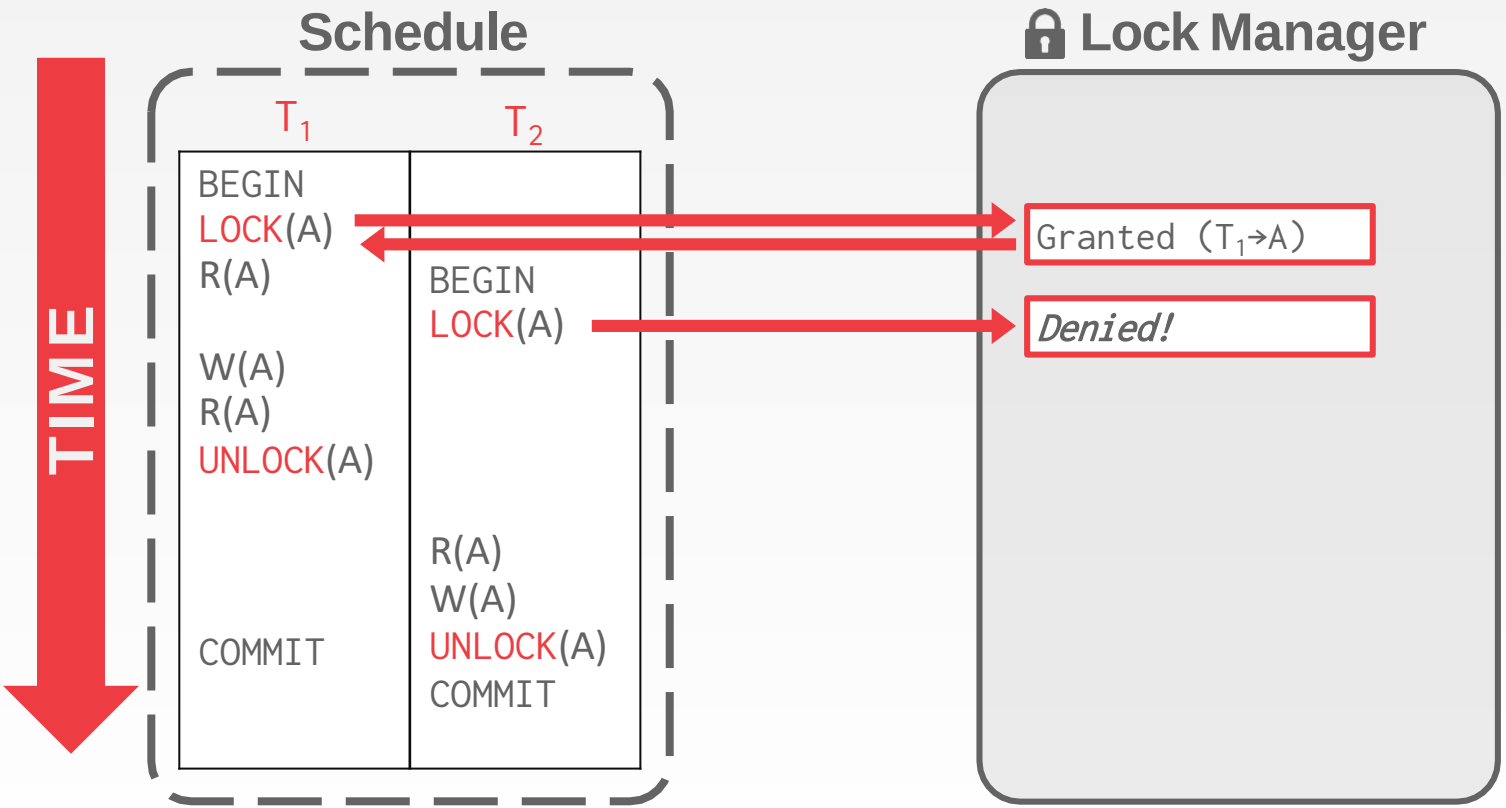
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS

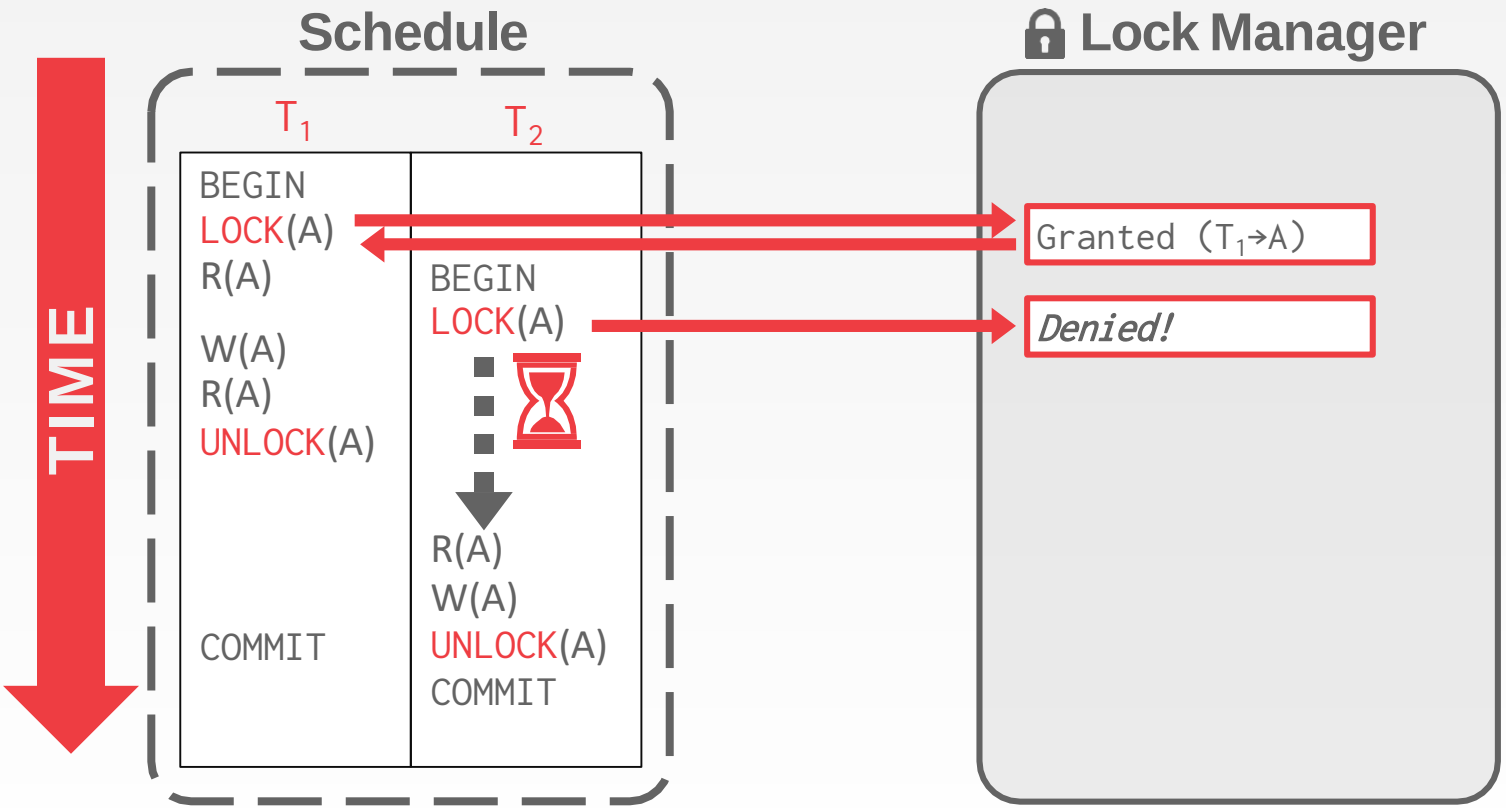


Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

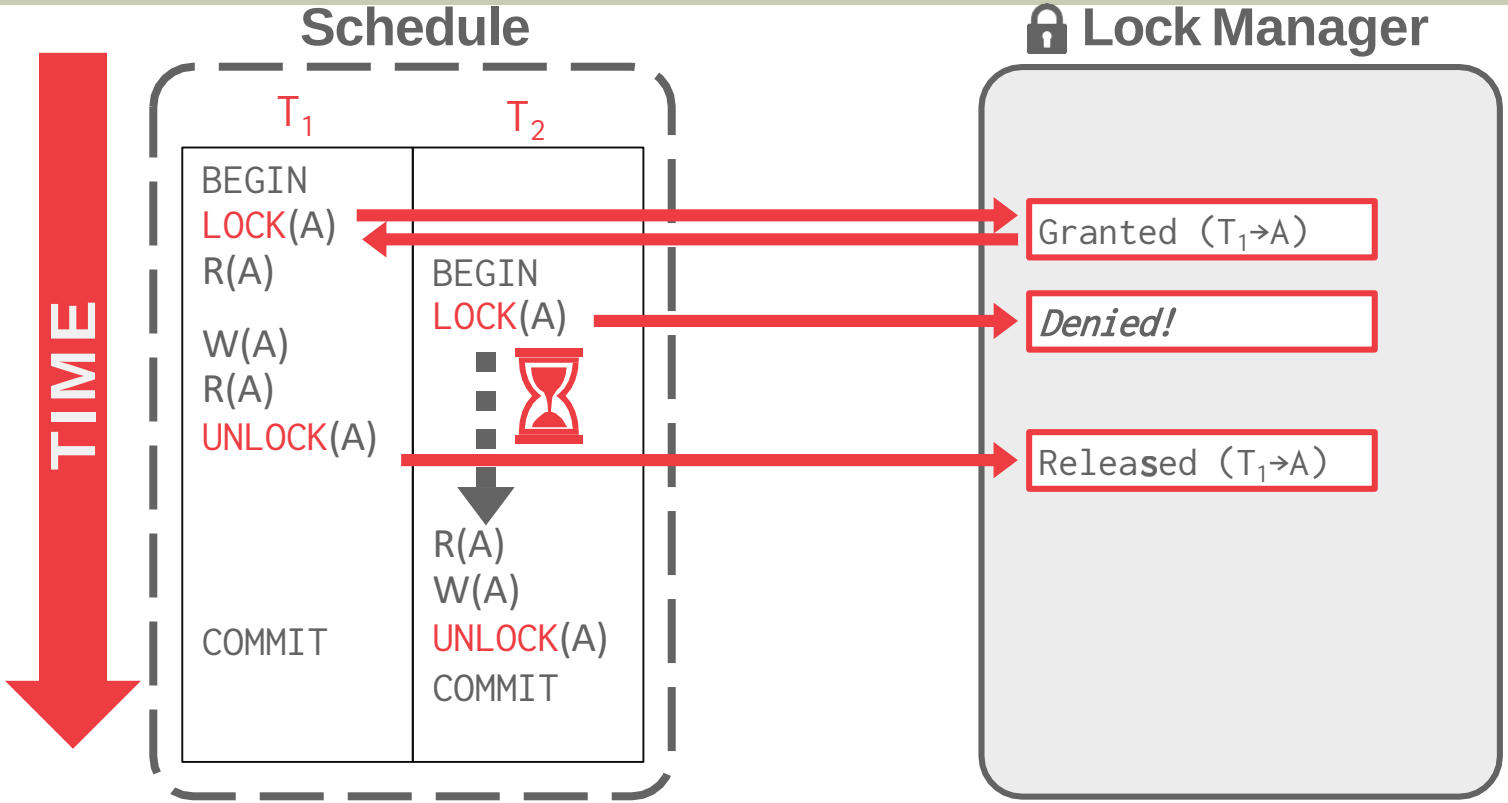
EXECUTING WITH LOCKS



EXECUTING WITH LOCKS

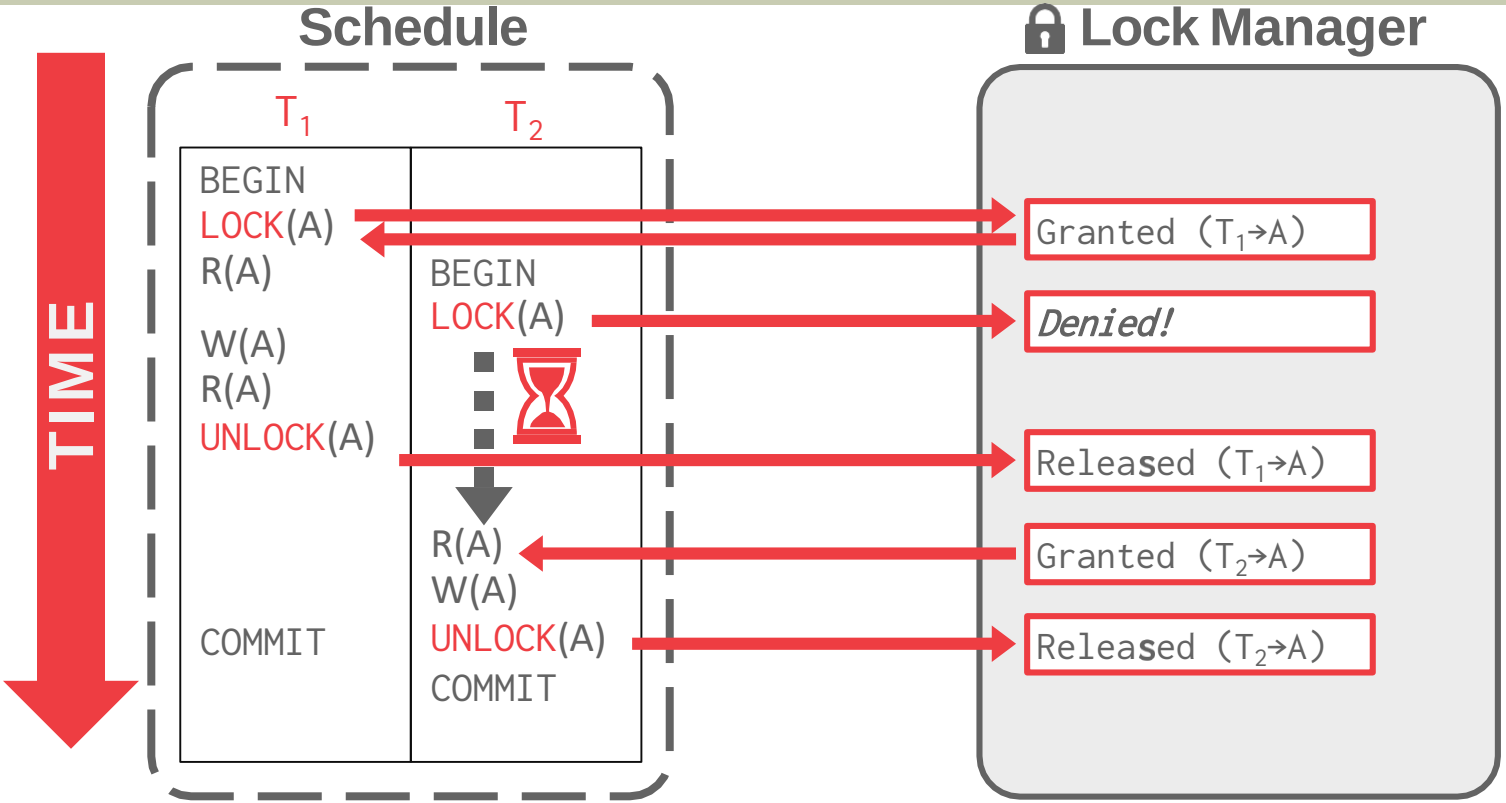


EXECUTING WITH LOCKS



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Outline

- Lock Types: Exclusive and Shared Locks
- Two-Phase Locking Protocol
- Executing Two-Phase Locking
- Strong Strict Two-phase Locking
- Non Two-phase Locking Example
- Example Two-Phase Locking with Deadlock
- Deadlock Detection

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

LOCKS VS. LATCHES

	Locks	Latches
Separate…	User transactions	Threads
Protect…	Database Contents	In-Memory Data Structures
During…	Entire Transactions	Critical Sections
Modes…	Shared, Exclusive, Update, Intention	Read, Write
Deadlock	Detection & Resolution	Avoidance
…by…	Waits-for, Timeout, Aborts	Coding Discipline
Kept in…	Lock Manager	Protected Data Structure

Source: Goetz Graefe

Locking-based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
  1. **exclusive** *(X) mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
  2. **shared** *(S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

BASIC LOCK TYPES

Lock compatibility matrix

	Shared	Exclusive
Shared	✓	X
Exclusive	X	X

S-LOCK: Shared locks for reads.

X-LOCK: Exclusive locks for writes.

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
- But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Shared and Exclusive Locks

- Shared/exclusive or read/write locks
  - Read operations on the same item are not conflicting
  - Must have exclusive lock to write
  - Three locking operations
    - shared_lock(X)
    - exclusive_lock(X)
    - unlock(X)

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Exclusive Lock Implementation

- Transaction requests access by issuing a lock_item(X) operation

```
lock_item(X):
B:  if LOCK(X) = 0              (*item is unlocked*)
        then LOCK(X) ←1         (*lock the item*)
    else
        begin
        wait (until LOCK(X) = 0
            and the lock manager wakes up the transaction);
        go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;                (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

Lock and unlock operations for exclusive locks

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Lock Table

- Lock table specifies items that have locks
- Lock manager subsystem
  - Keeps track of and controls access to locks
  - Rules enforced by lock manager module
- At most one transaction can hold the lock on an item at a given time
- Binary locking too restrictive for database items

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Shared and Exclusive Locks

```
read_lock(X):
B:  if LOCK(X) = "unlocked"
        then begin LOCK(X) ← "read-locked";
            no_of_reads(X) ← 1
            end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
            end;
write_lock(X):
B:  if LOCK(X) = "unlocked"
        then LOCK(X) ← "write-locked"
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
            end;
unlock (X):
    if LOCK(X) = "write-locked"
        then begin LOCK(X) ← "unlocked";
                wakeup one of the waiting transactions, if any
                end
    else it LOCK(X) = "read-locked"
        then begin
                no_of_reads(X) ← no_of_reads(X) −1;
                if no_of_reads(X) = 0
                    then begin LOCK(X) = "unlocked";
                            wakeup one of the waiting transactions, if any
                            end
                end;
```

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS

Transactions request locks (or upgrades).

Lock manager grants or blocks requests.

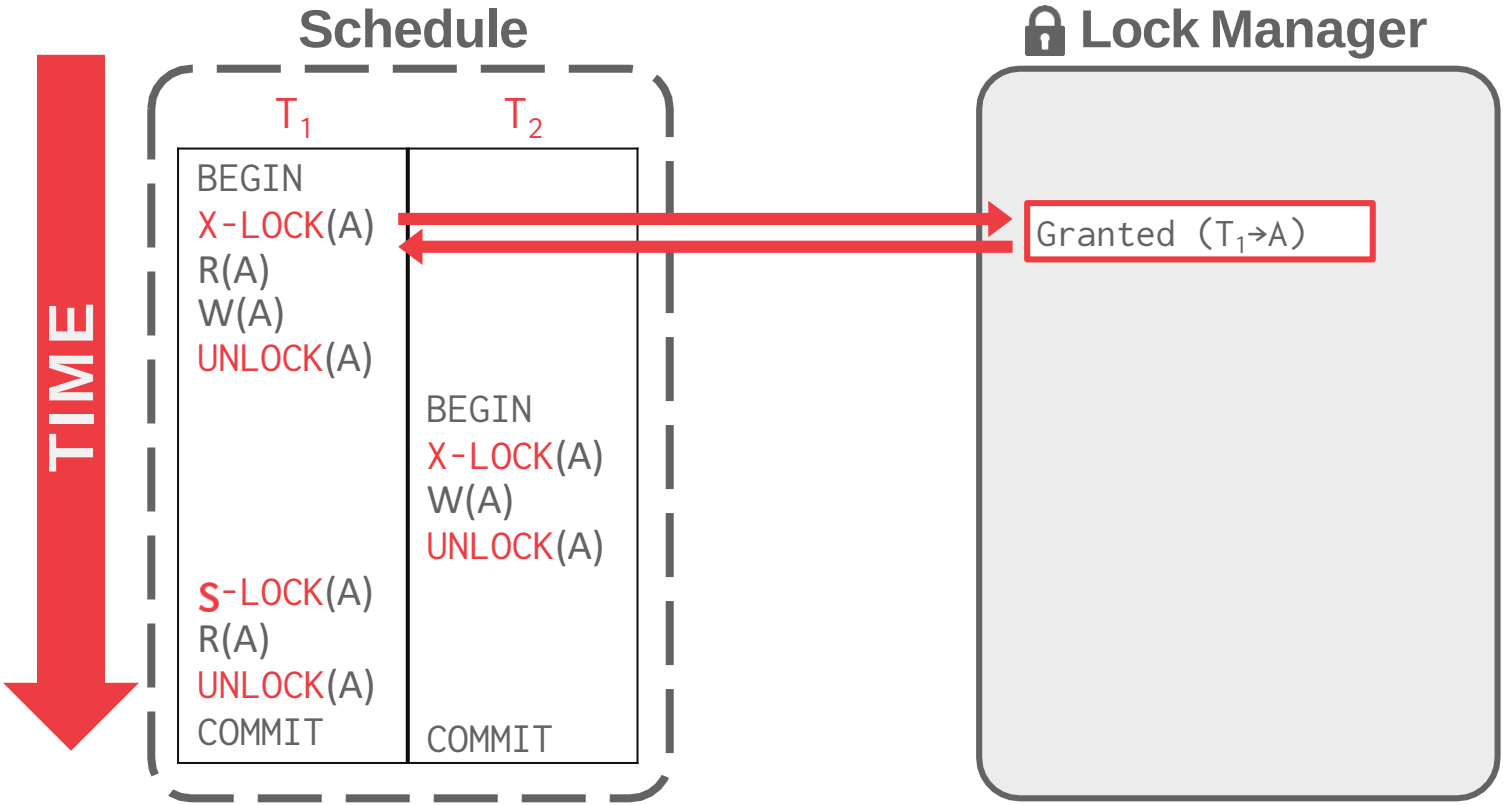
Transactions release locks.

Lock manager updates its internal lock-table.

→ It keeps track of what transactions hold what locks and what transactions are waiting to acquire any locks.

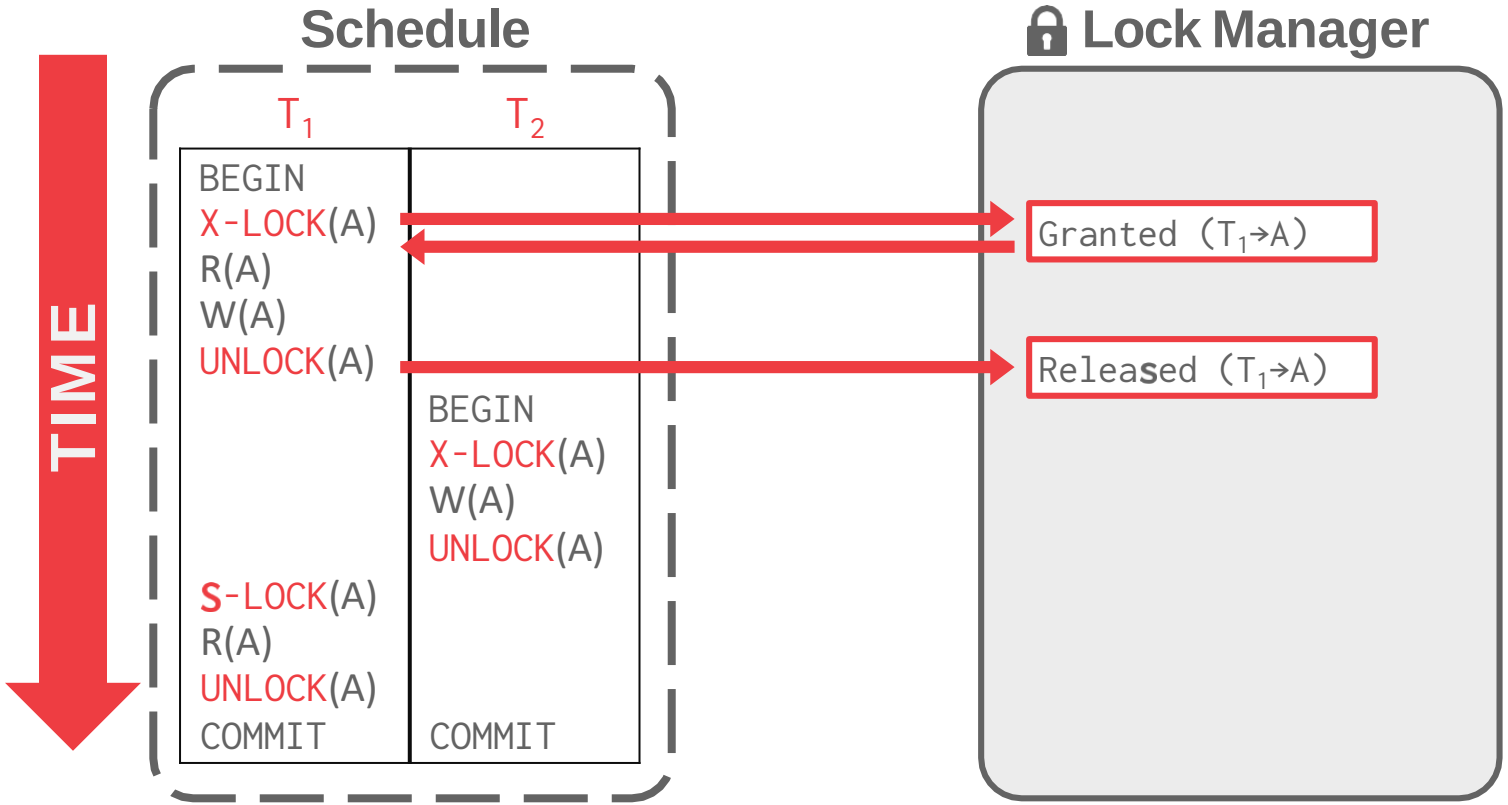
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS



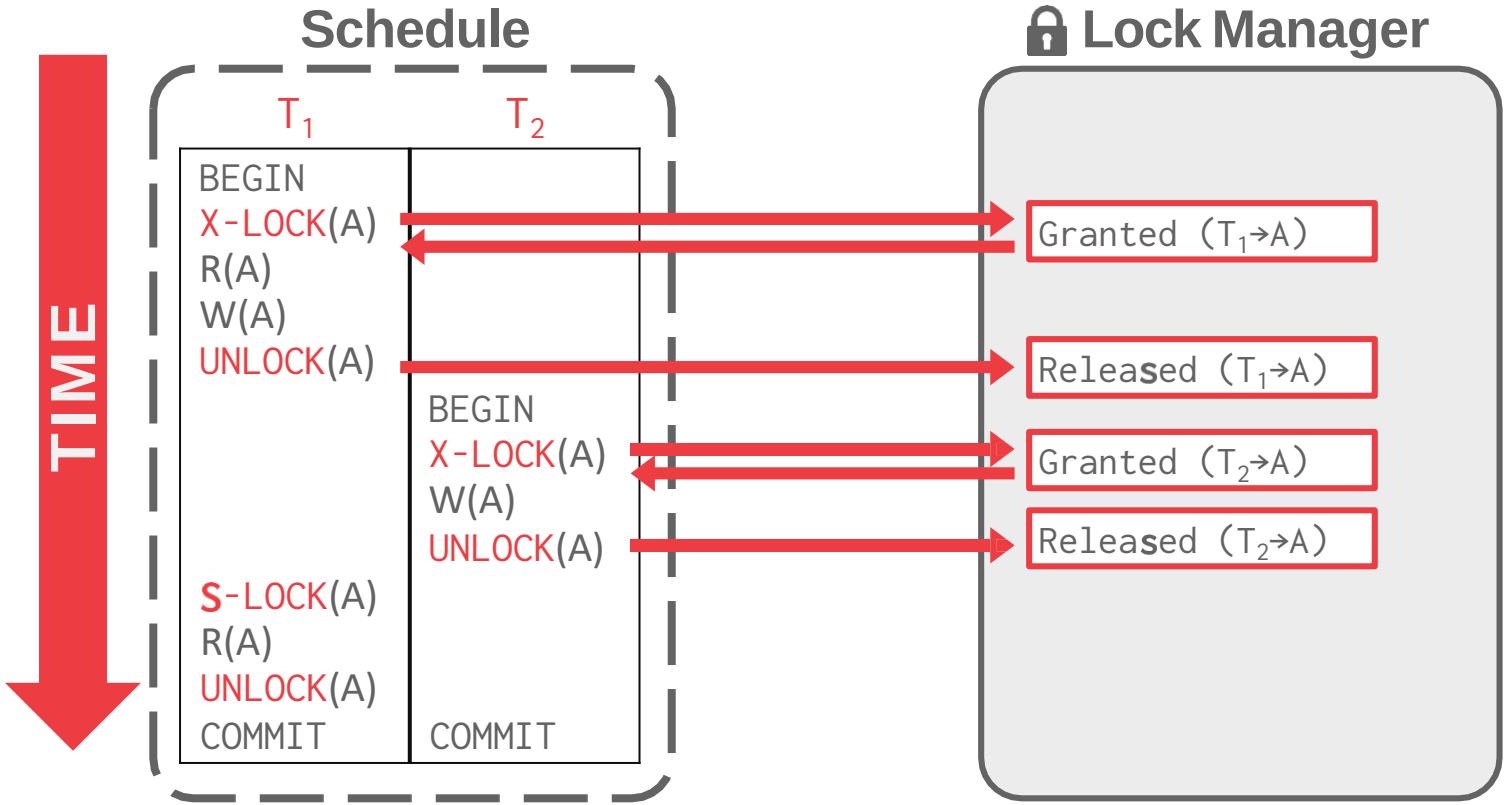
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS



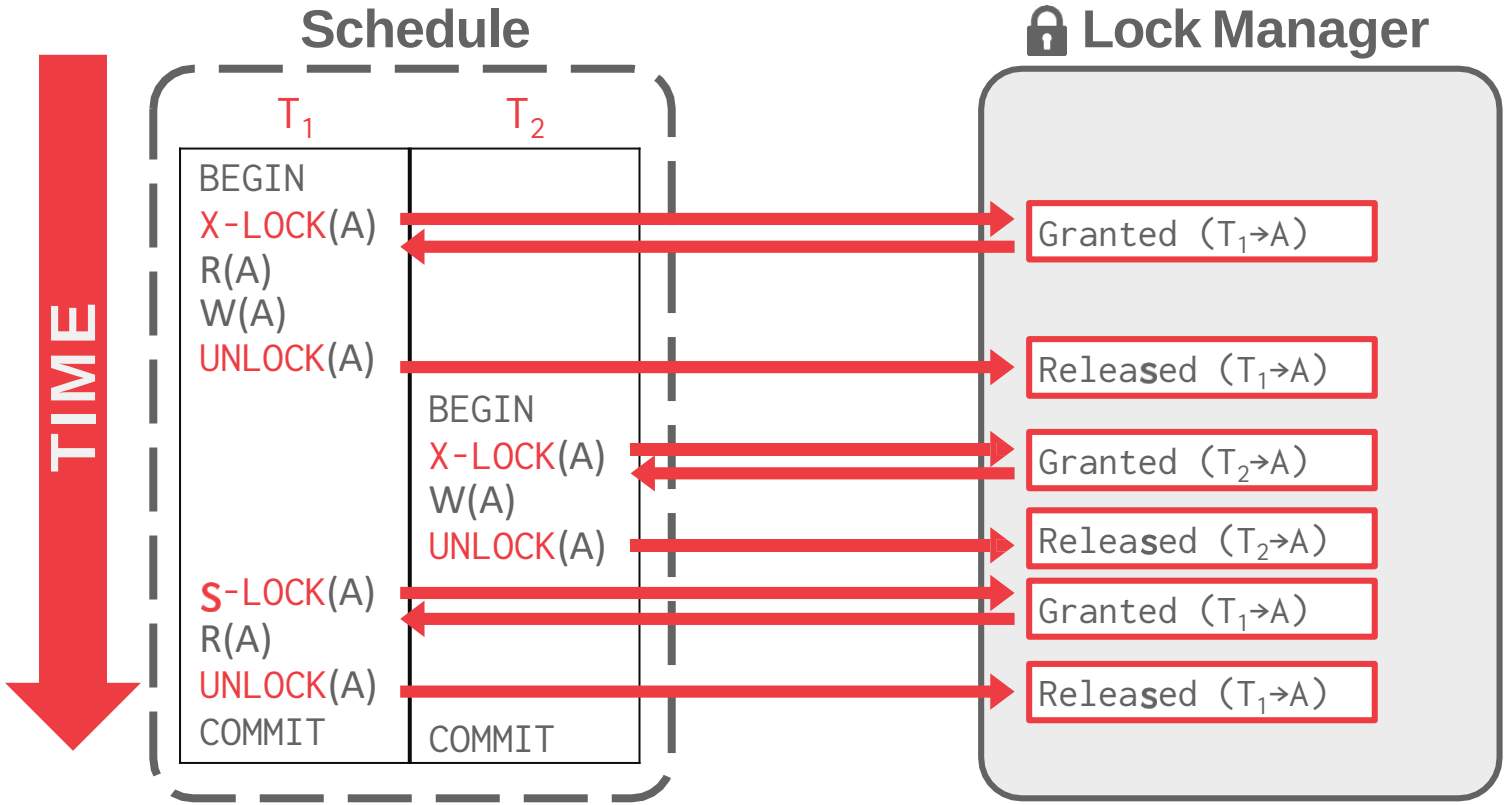
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS



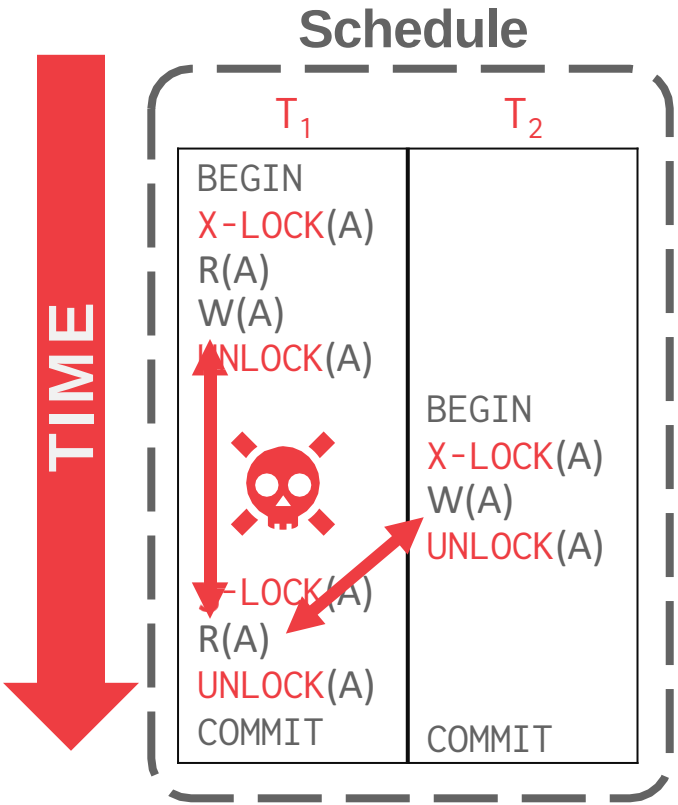
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

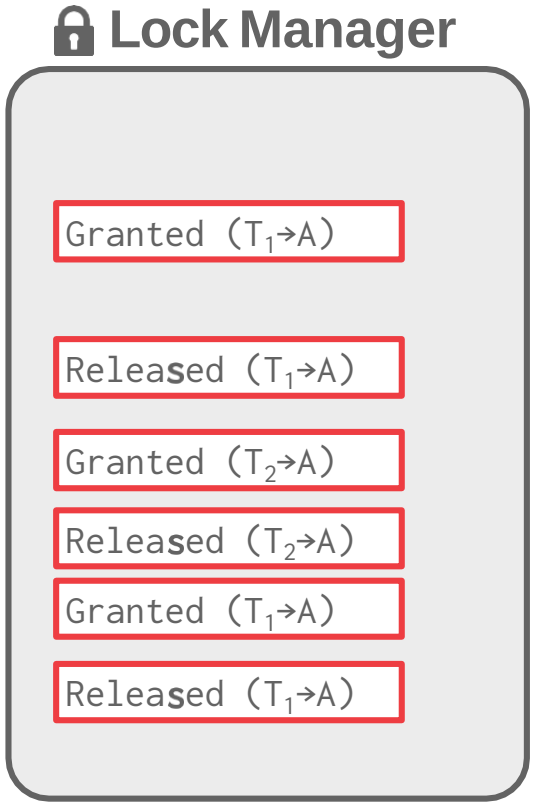
EXECUTING WITH LOCKS



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH LOCKS





Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Concurrency Control Protocol

Two-phase locking (2PL) is a concurrency control protocol that determines whether a txn can access an object in the database on the fly.

The protocol does <u>not</u> need to know all the queries that a txn will execute ahead of time.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Two-Phase Locking

Phase #1: Growing

→ Each txn requests the locks that it needs from the DBMS's lock manager.

→ The lock manager grants/denies lock requests.

Phase #2: Shrinking

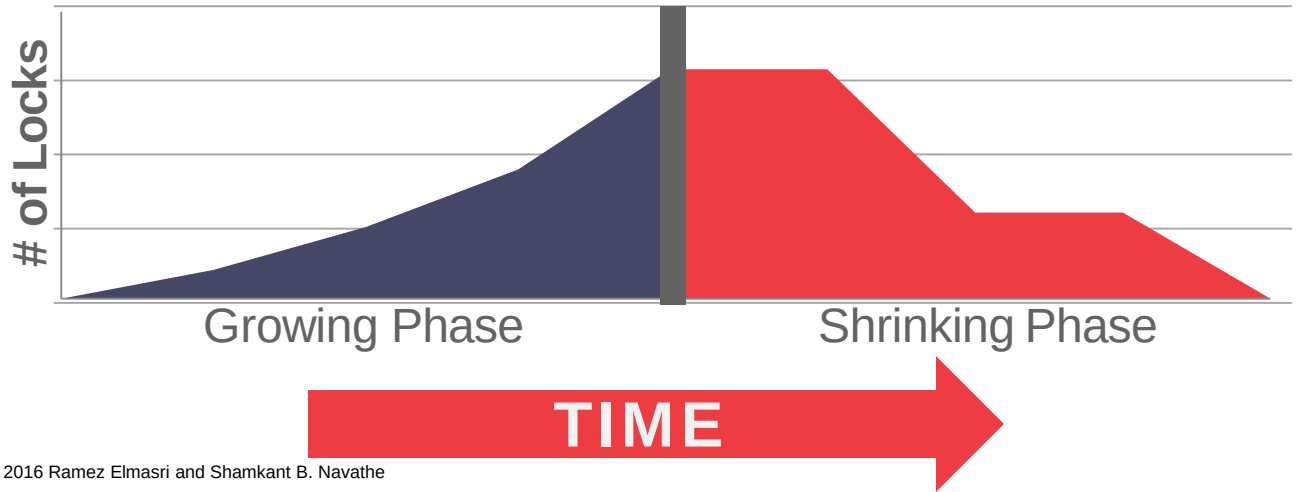
→ The txn is allowed to only release locks that it previously acquired. It cannot acquire new locks.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Two-Phase Locking

The txn is not allowed to acquire/upgrade locks after the growing phase finishes.

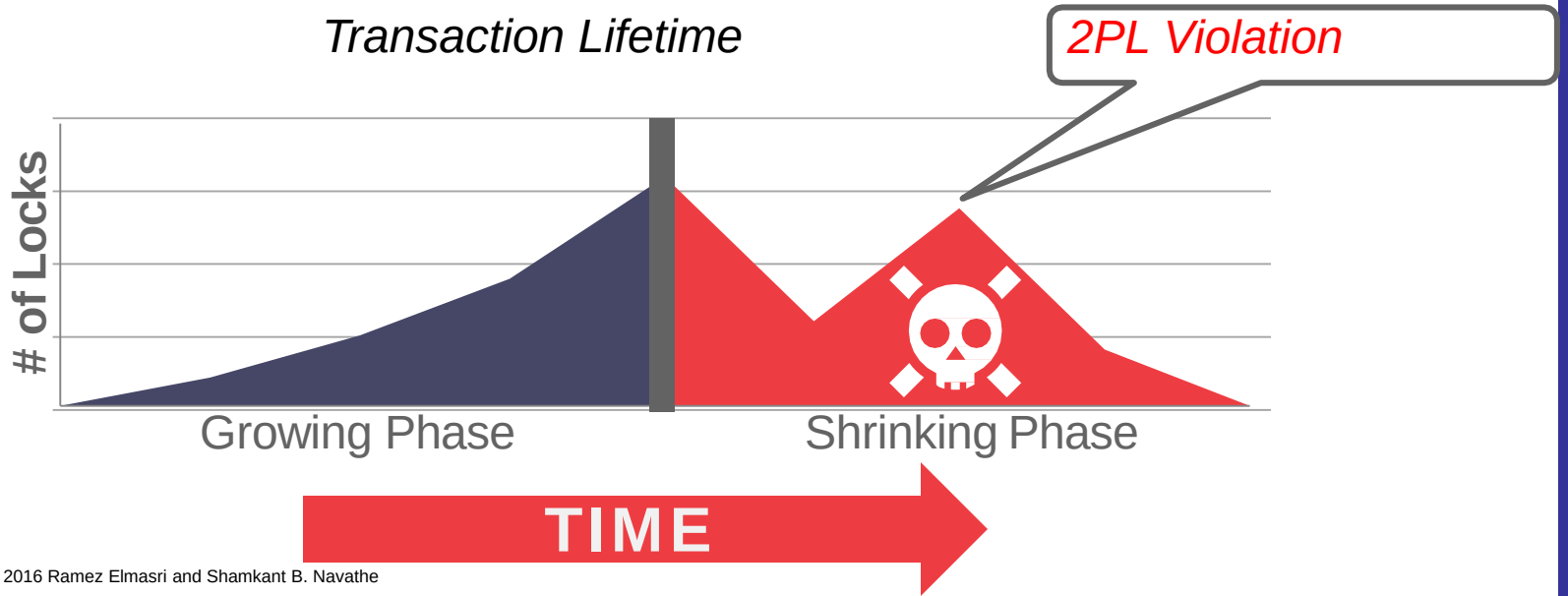
Transaction Lifetime



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

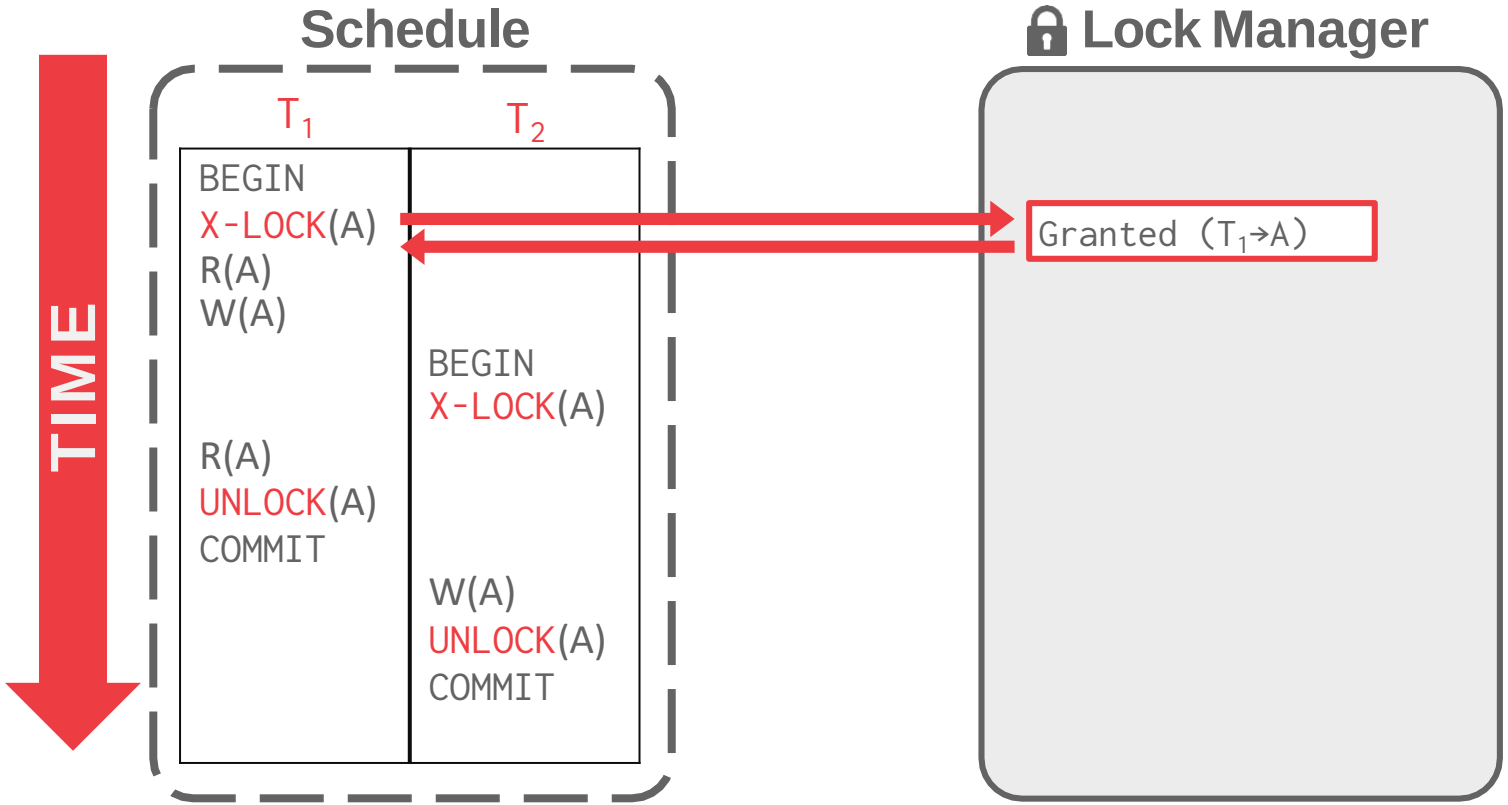
Two-Phase Locking

The txn is not allowed to acquire/upgrade locks after the growing phase finishes.



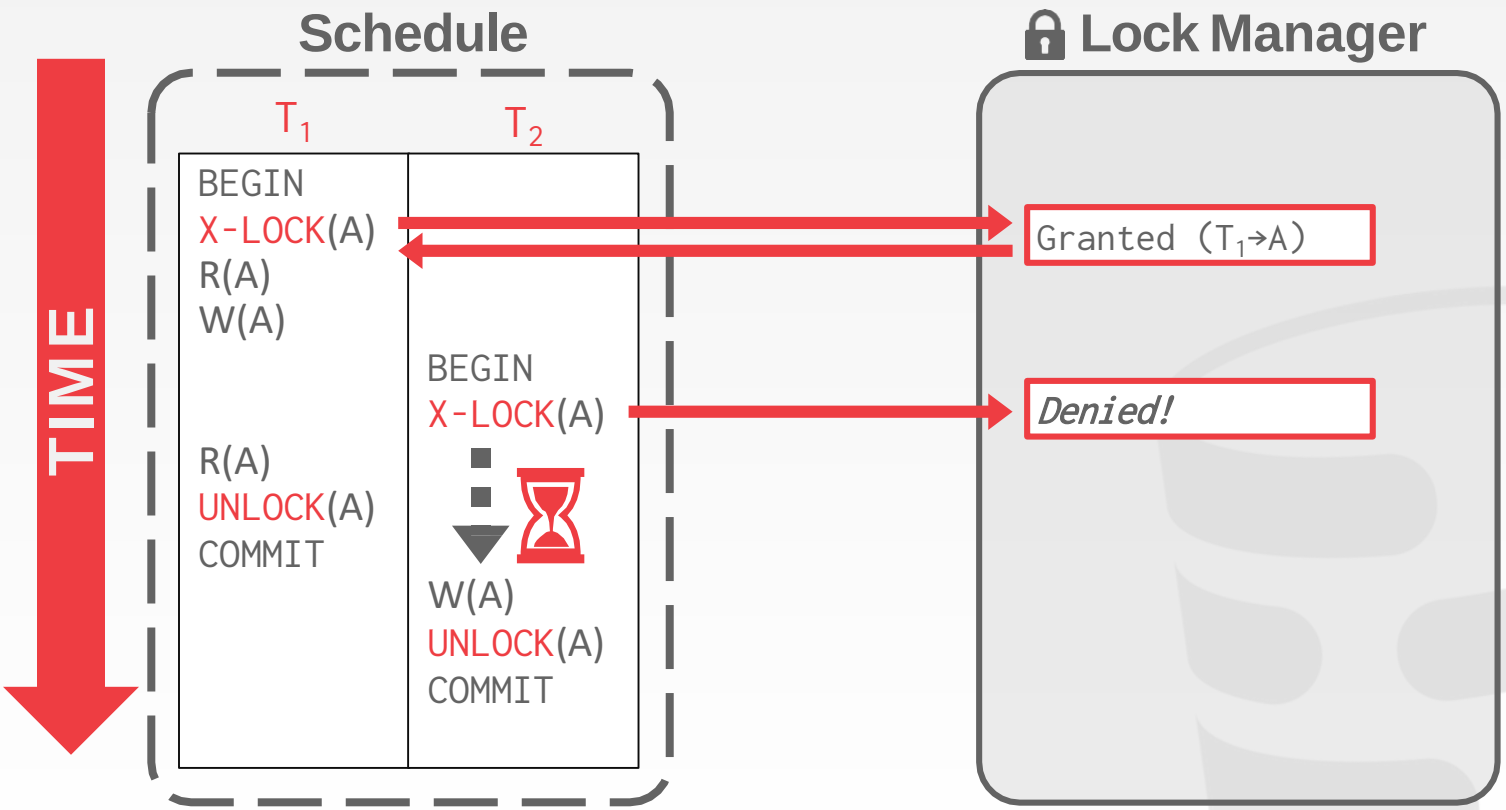
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH 2PL

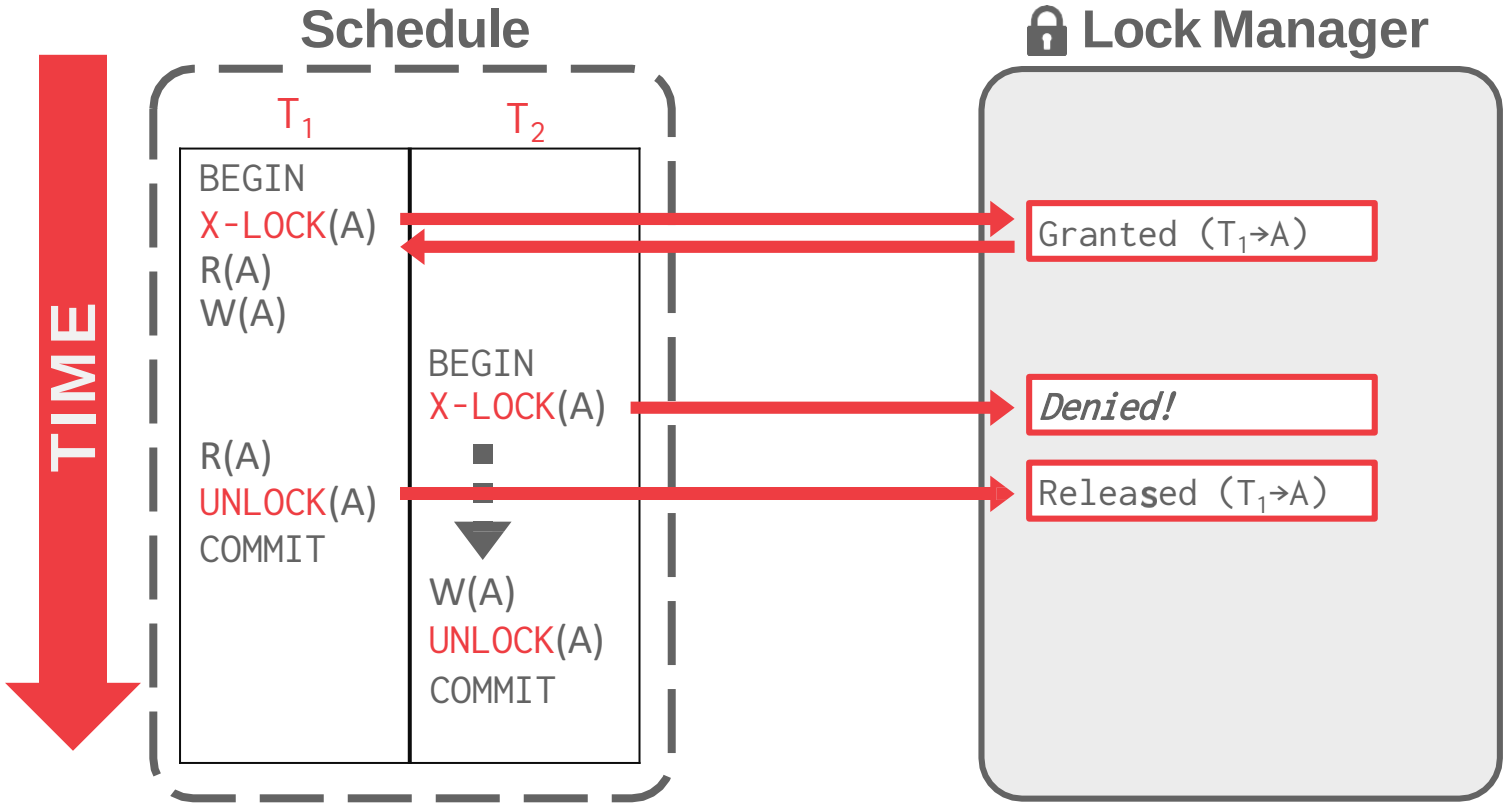


Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH 2PL

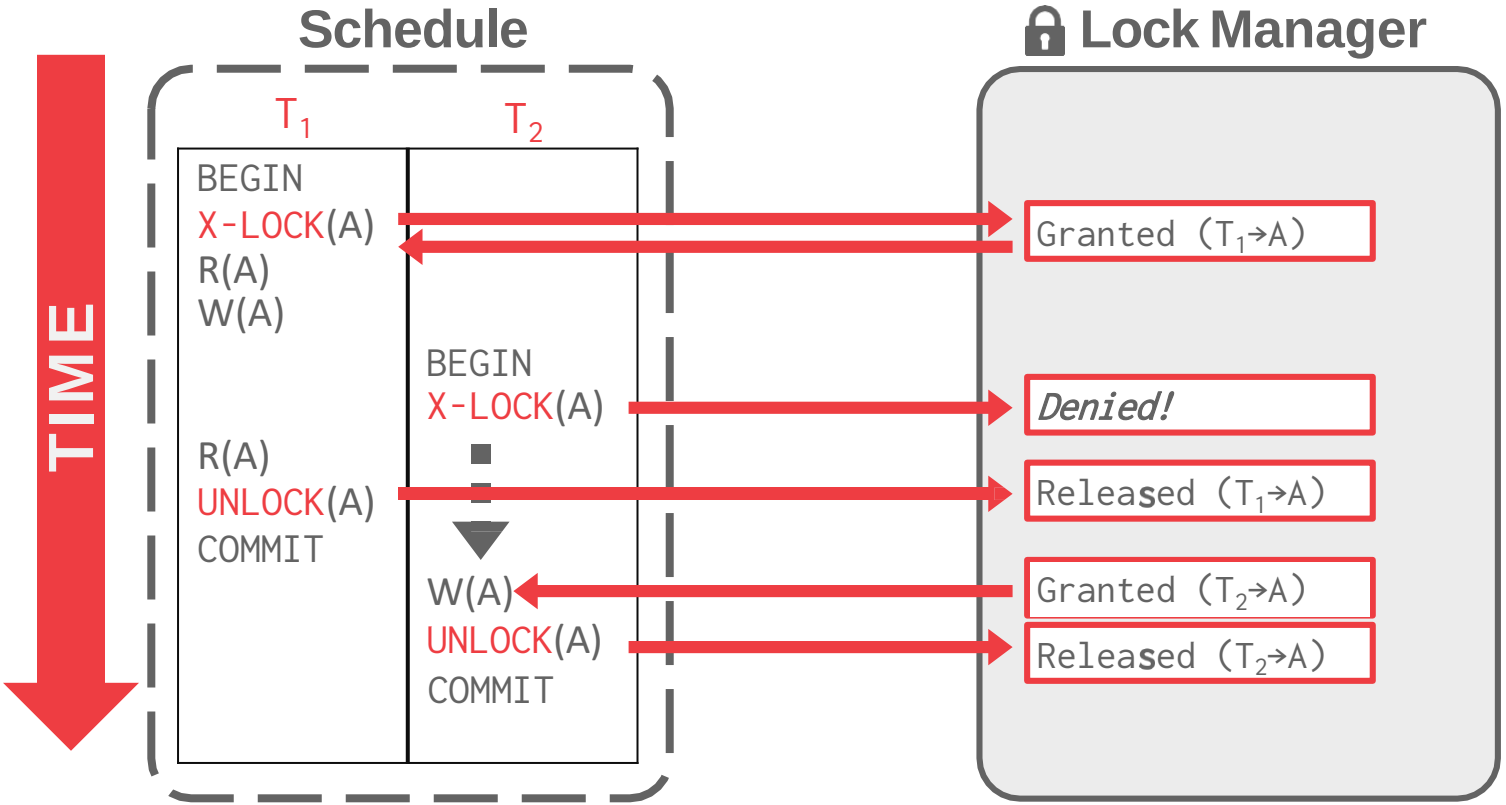


EXECUTING WITH 2PL



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXECUTING WITH 2PL



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Outline

- Strong Strict Two-phase Locking
- Example Two-Phase Locking with Deadlock
- Deadlock
- Deadlock Detection
- Deadlock Detection : Victim Selection
- Deadlock Prevention: Wait-die and Wound-Wait
- Lock Granularity and Database Lock Hierarchy
- Multi-granularity Locks

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Two-Phase Locking

2PL on its own is sufficient to guarantee conflict serializability.

→ It generates schedules whose precedence graph is acyclic.

But it is subject to **cascading aborts**.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL – Cascading Aborts

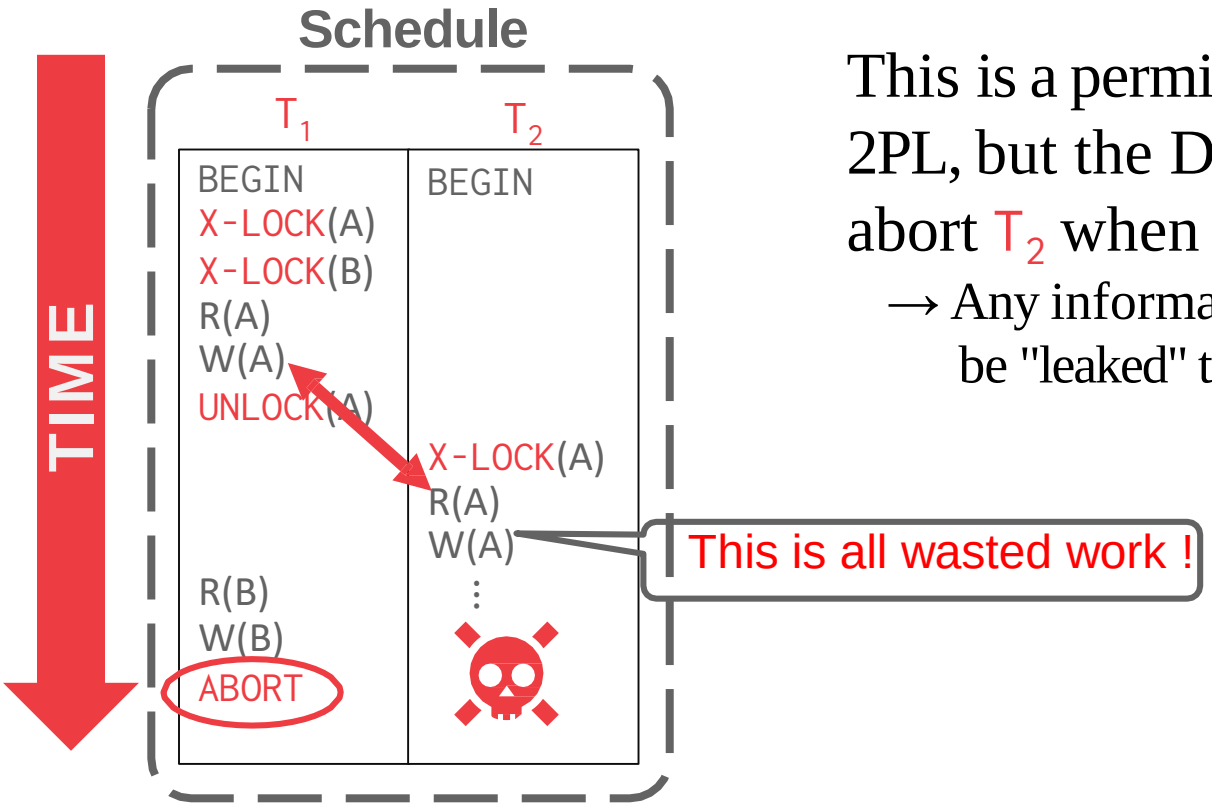
Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
X-LOCK(B)	
R(A)	
W(A)	
UNLOCK(A)	
	X-LOCK(A)
	R(A)
	W(A)
R(B)	⋮
W(B)	
ABORT	

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL – CASCADING ABORTS



This is a permissible schedule in 2PL, but the DBMS has to also abort T_2 when T_1 aborts.

→ Any information about T_1 cannot be "leaked" to the outside world.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL OBSERVATIONS

There are potential schedules that are serializable but would not be allowed by 2PL.
→ Locking limits concurrency.

May still have "dirty reads".
→ Solution: **Strong Strict 2PL (aka Rigorous 2PL)**

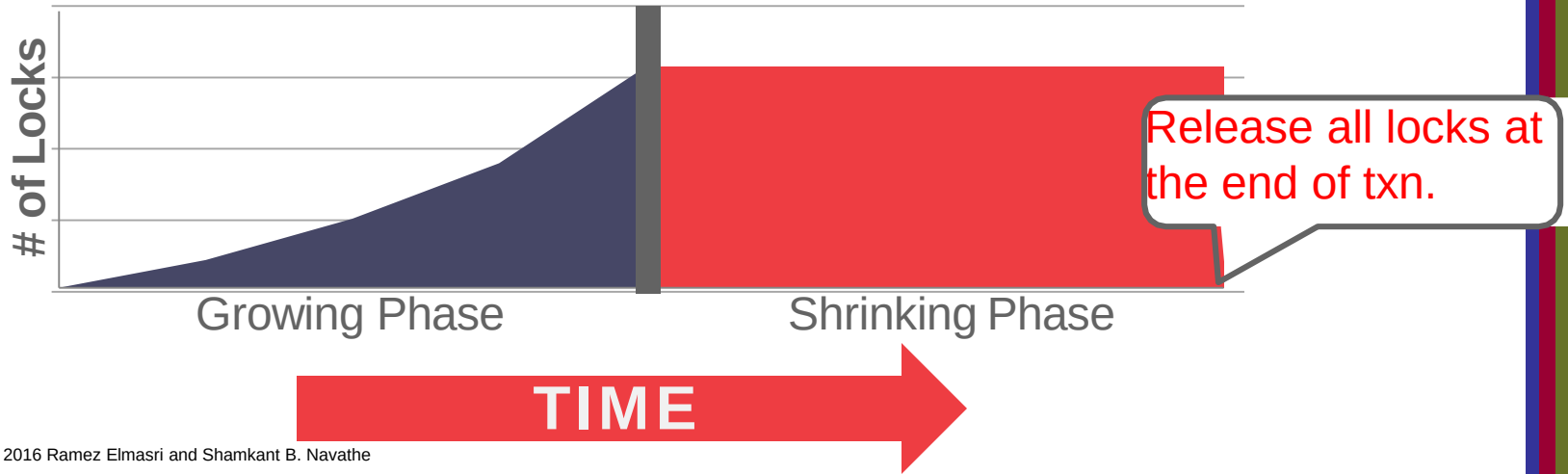
May lead to deadlocks.
→ Solution: **Detection** or **Prevention**

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

STRONG STRICT TWO-PHASE LOCKING

The txn is not allowed to acquire/upgrade locks after the growing phase finishes.

Allows only conflict serializable schedules, but it is often stronger than needed for some apps.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Strong Strict Two-Phase Locking

A schedule is **strict** if a value written by a txn is not read or overwritten by other txns until that txn finishes.

Advantages:

→ Does not incur cascading aborts.

→ Aborted txns can be undone by just restoring original values of modified tuples.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXAMPLES

T_1 – Move $100 from Andy's account (A) to his bookie's account (B).

T_2 – Compute the total amount in all accounts and return it to the application.

T_1

```
BEGIN
A=A-100
B=B+100
COMMIT
```

T_2

```
BEGIN
ECHO A+B
COMMIT
```

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Non-2 PL Example

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
	S-LOCK(A)
A=A-100	
W(A)	
UNLOCK(A)	
	R(A)
	UNLOCK(A)
	S-LOCK(B)
X-LOCK(B)	
	R(B)
	UNLOCK(B)
	ECHO A+B
R(B)	COMMIT
B=B+100	
W(B)	
UNLOCK(B)	
COMMIT	

Initial Database State

A=1000, B=1000

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

NON-2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
	S-LOCK(A)
A=A-100	
W(A)	
UNLOCK(A)	
	R(A)
	UNLOCK(A)
	S-LOCK(B)
X-LOCK(B)	
	R(B)
	UNLOCK(B)
	ECHO A+B
R(B)	
B=B+100	
W(B)	
UNLOCK(B)	
COMMIT	
	COMMIT

Initial Database State

A=1000, B=1000

NON-2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
	S-LOCK(A)
A=A-100	
W(A)	
UNLOCK(A)	
	R(A)
	UNLOCK(A)
	S-LOCK(B)
X-LOCK(B)	
	R(B)
R(B)	UNLOCK(B)
B=B+100	ECHO A+B
W(B)	COMMIT
UNLOCK(B)	
COMMIT	

Initial Database State

A=1000, B=1000

NON-2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
A=A-100	
W(A)	
UNLOCK(A)	
	S-LOCK(A)
	R(A)
	UNLOCK(A)
	S-LOCK(B)
X-LOCK(B)	
	R(B)
	UNLOCK(B)
	ECHO A+B
	COMMIT
R(B)	
B=B+100	
W(B)	
UNLOCK(B)	
COMMIT	

Initial Database State

A=1000, B=1000

T_2 Output

A+B=1900

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	S-LOCK(A)
A=A-100	
W(A)	
X-LOCK(B)	
UNLOCK(A)	R(A)
	S-LOCK(B)
R(B)	
B=B+100	
W(B)	
UNLOCK(B)	R(B)
COMMIT	UNLOCK(A)
	UNLOCK(B)
	ECHO A+B
	COMMIT

Initial Database State

A=1000, B=1000

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
	S-LOCK(A)
A=A-100	
W(A)	
X-LOCK(B)	
UNLOCK(A)	
	R(A)
	S-LOCK(B)
R(B)	
B=B+100	
W(B)	
UNLOCK(B)	
COMMIT	
	R(B)
	UNLOCK(A)
	UNLOCK(B)
	ECHO A+B
	COMMIT

Initial Database State

A=1000, B=1000

2PL EXAMPLE

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
	S-LOCK(A)
A=A-100	
W(A)	
X-LOCK(B)	
UNLOCK(A)	
	R(A)
	S-LOCK(B)
R(B)	
B=B+100	
W(B)	
UNLOCK(B)	
COMMIT	
	R(B)
	UNLOCK(A)
	UNLOCK(B)
	ECHO A+B
	COMMIT

Initial Database State

A=1000, B=1000

T_2 Output

A+B=2000

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Strong Strict 2PL EXAMPLE

TIME

Schedule

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	S-LOCK(A)
A=A-100	
W(A)	
X-LOCK(B)	
R(B)	
B=B+100	
W(B)	
UNLOCK(A)	
UNLOCK(B)	R(A)
COMMIT	S-LOCK(B)
	R(B)
	ECHO A+B
	UNLOCK(A)
	UNLOCK(B)
	COMMIT

Initial Database State

A=1000, B=1000

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Strong Strict Two-Phase Locking Example

Schedule

TIME

T_1	T_2
BEGIN	BEGIN
X-LOCK(A)	
R(A)	
A=A-100	
W(A)	
X-LOCK(B)	
R(B)	
B=B+100	
W(B)	
UNLOCK(A)	
UNLOCK(B)	
COMMIT	
	S-LOCK(A)
	R(A)
	S-LOCK(B)
	R(B)
	ECHO A+B
	UNLOCK(A)
	UNLOCK(B)
	COMMIT

Initial Database State

A=1000, B=1000

T_2 Output

A+B=2000

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

2PL OBSERVATIONS

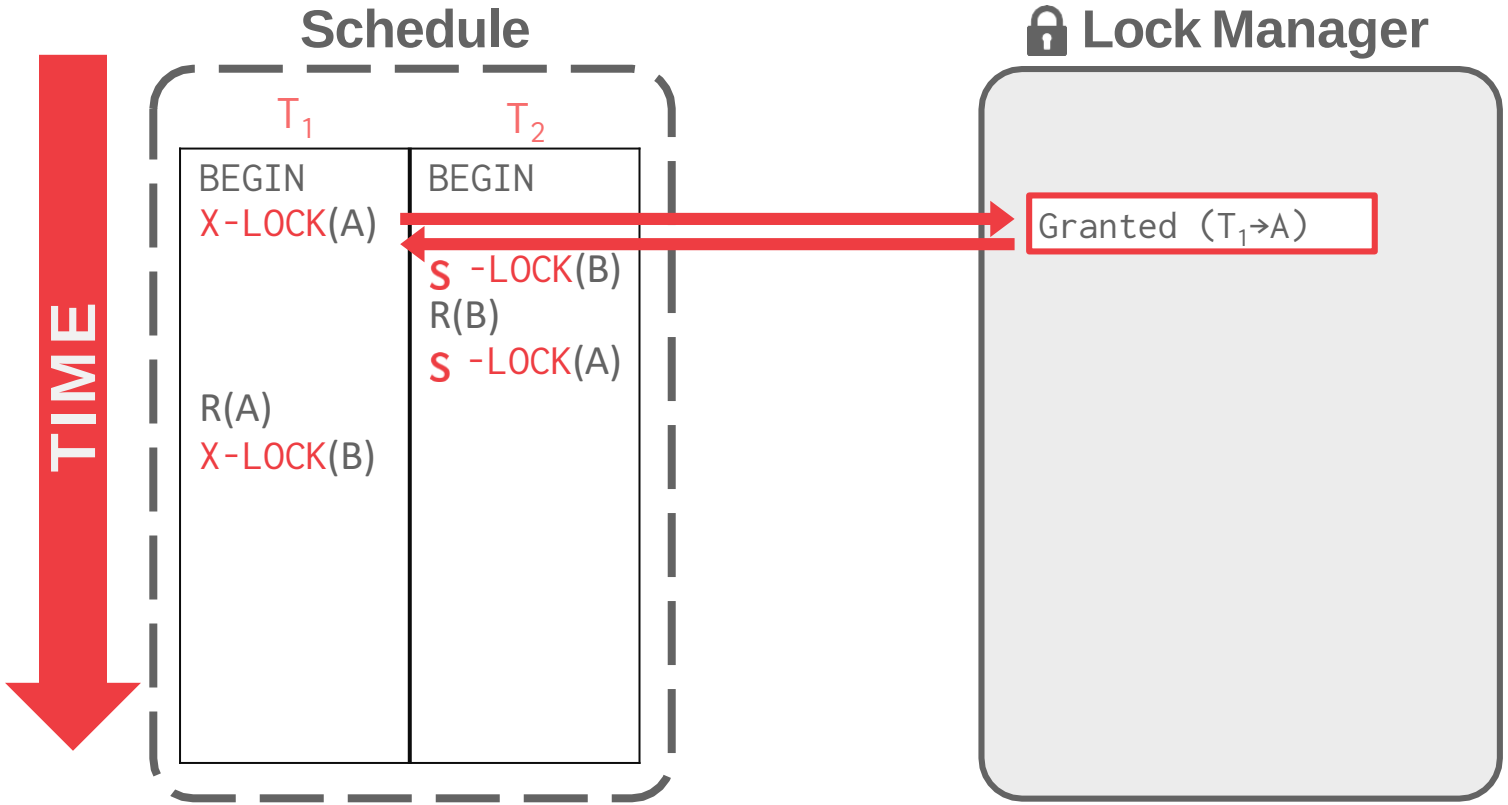
There are potential schedules that are serializable but would not be allowed by 2PL.
→ Locking limits concurrency.

May still have "dirty reads".
→ Solution: **Strong Strict 2PL (Rigorous)**

May lead to deadlocks.
→ Solution: **Detection** or **Prevention**

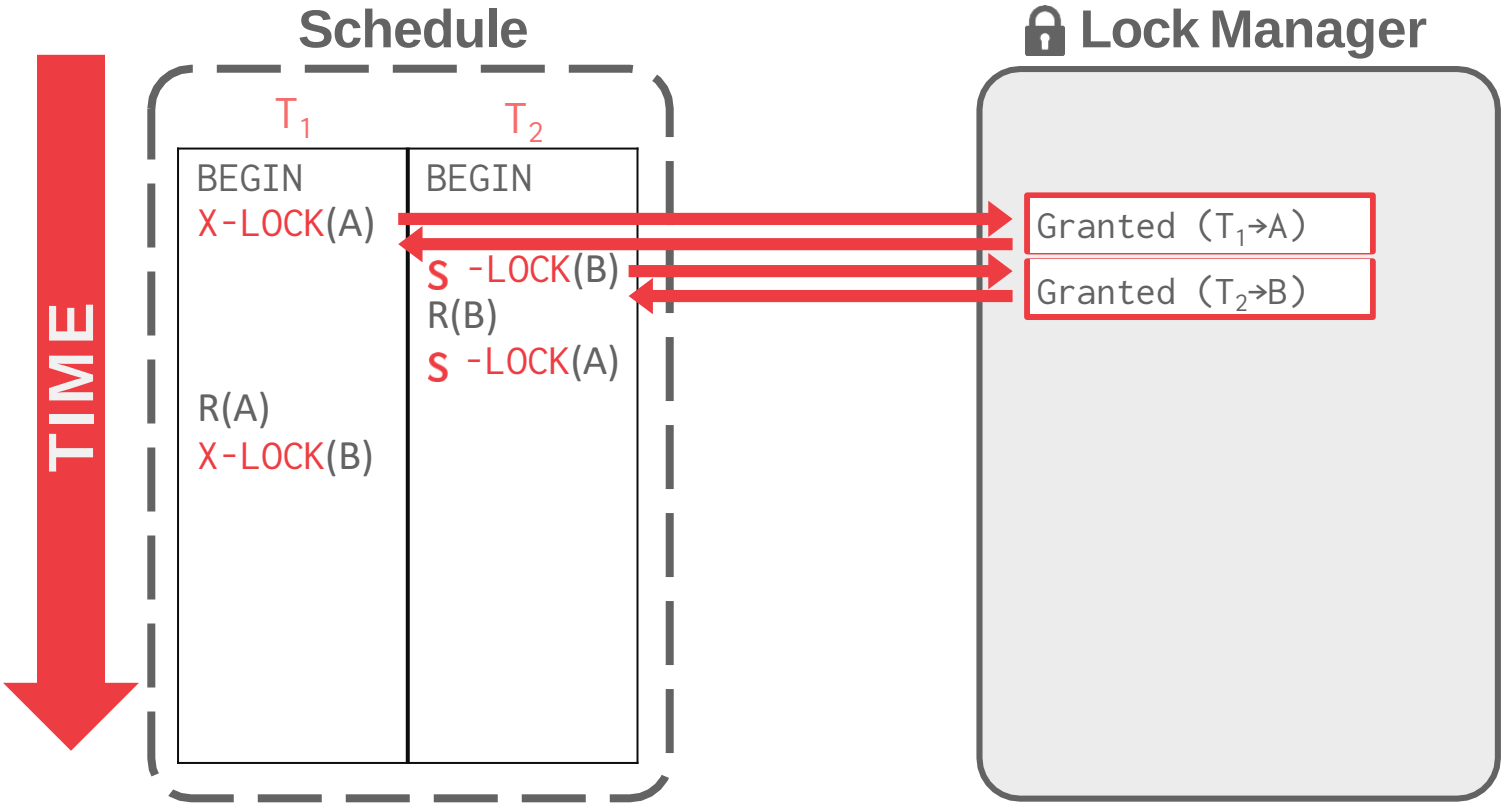
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL



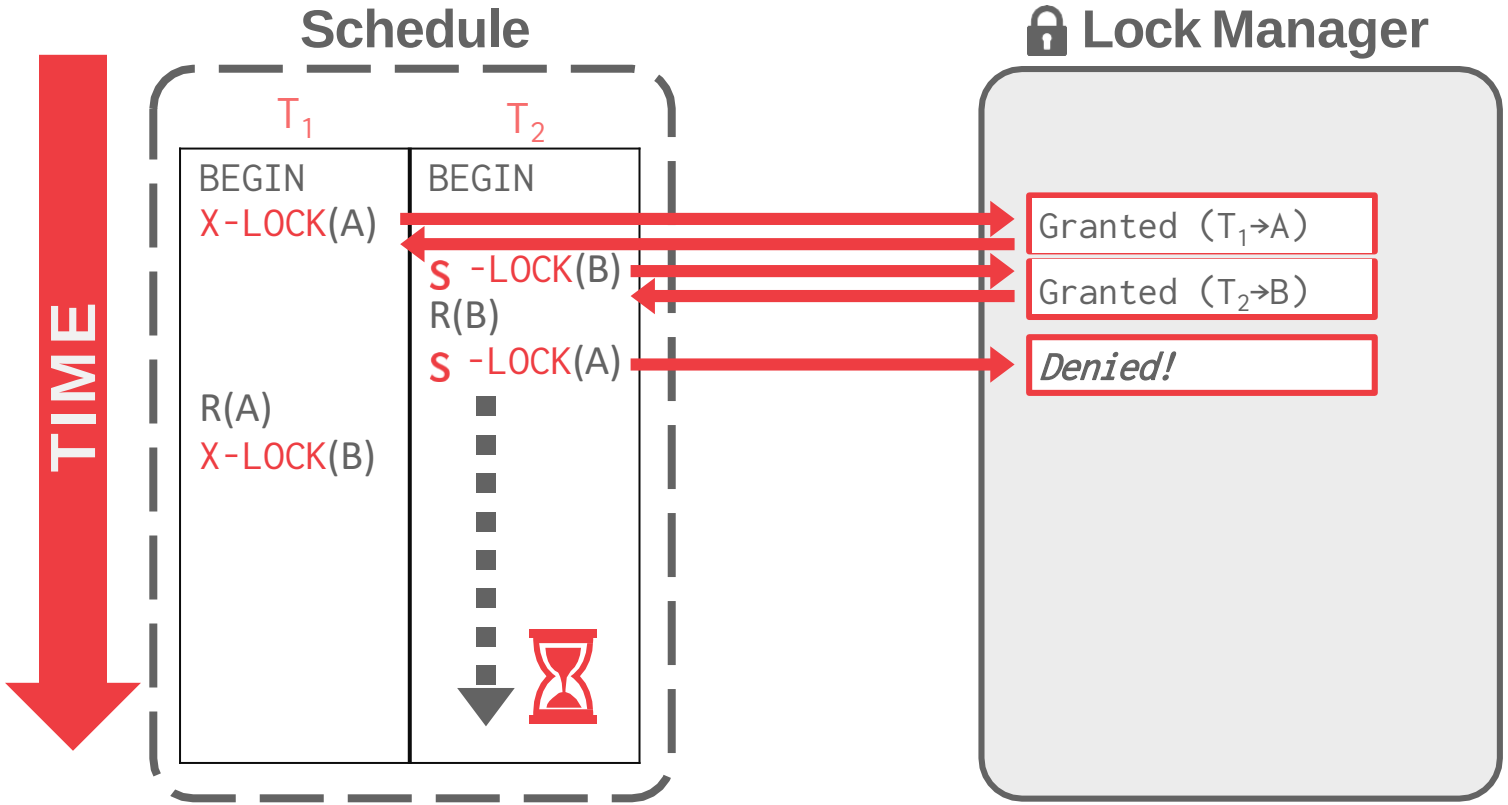
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL



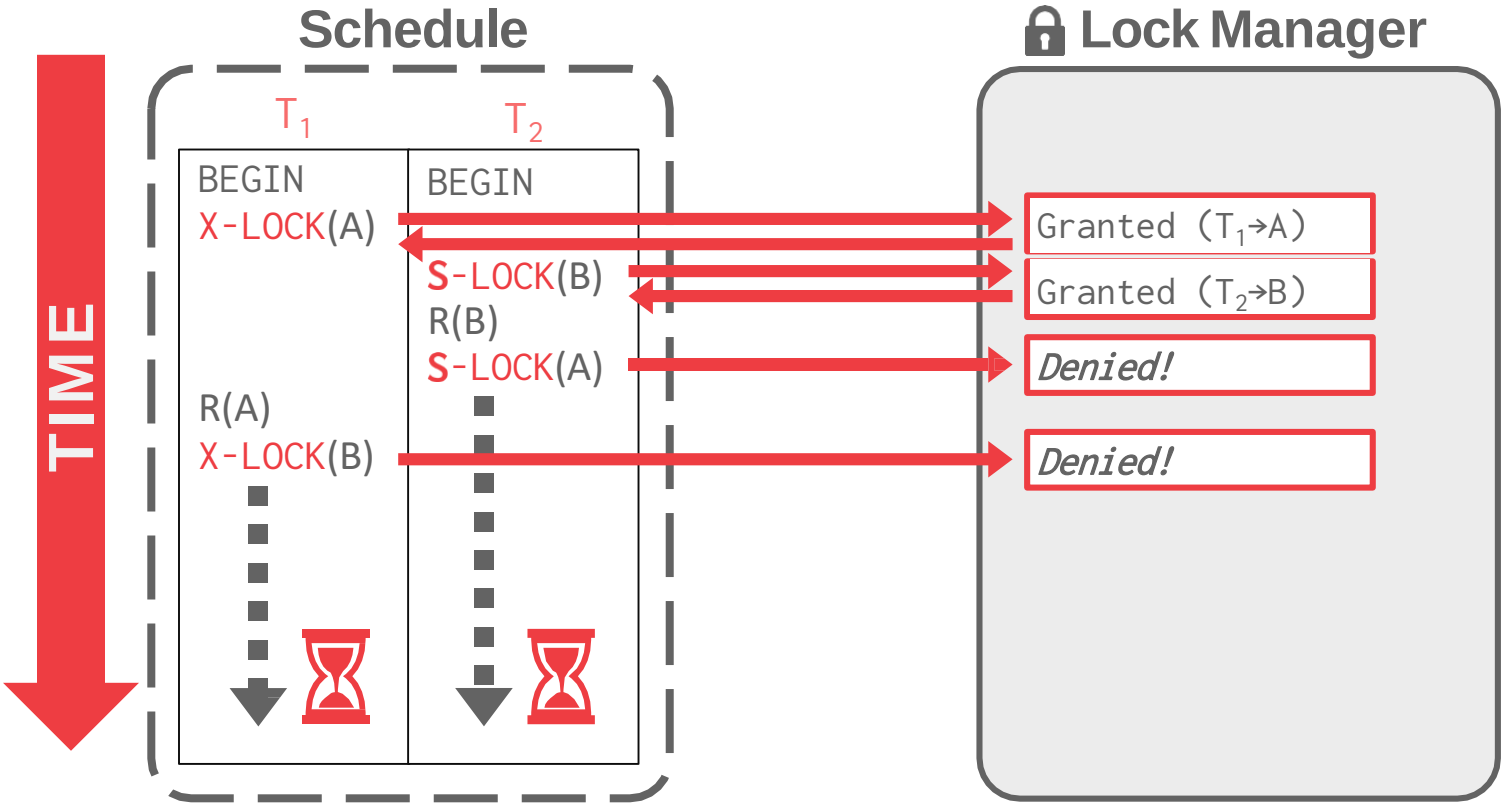
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL



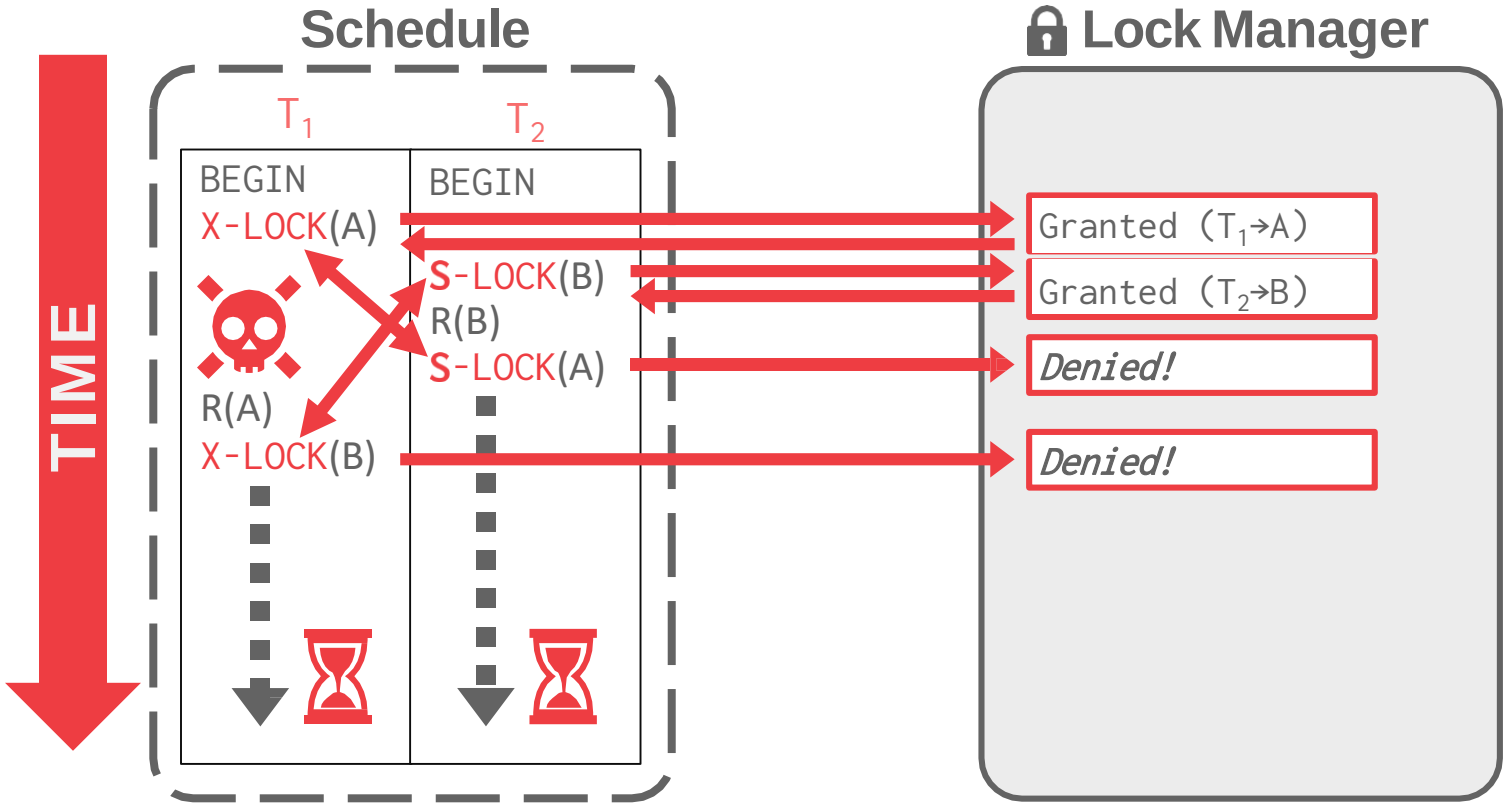
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock in 2PL

A **deadlock** is a cycle of transactions waiting for locks to be released by each other.

Two ways of dealing with deadlocks:

→ **Approach #1: Deadlock Detection**

→ **Approach #2: Deadlock Prevention**

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Detection

The DBMS creates a **waits-for** graph to keep track of what locks each txn is waiting to acquire:

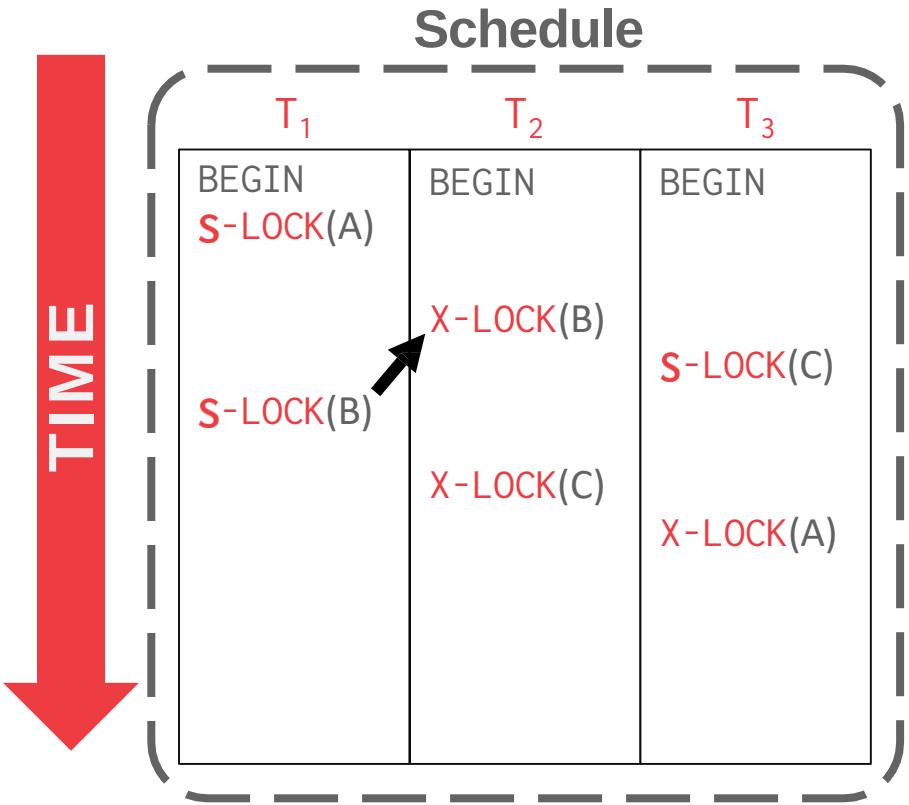
→ Nodes are transactions

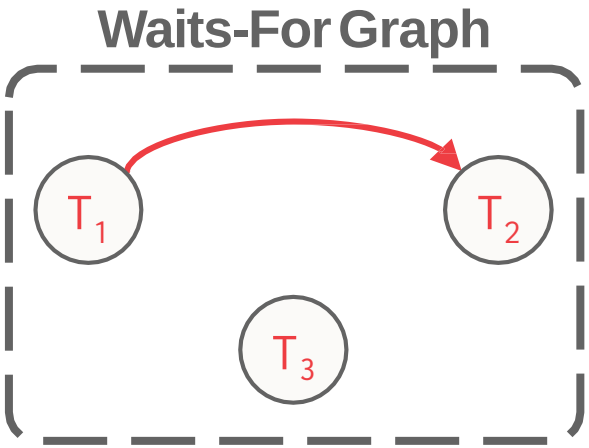
→ Edge from T_i to T_j if T_i is waiting for T_j to release a lock.

The system periodically checks for cycles in **waits-for** graph and then decides how to break it.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

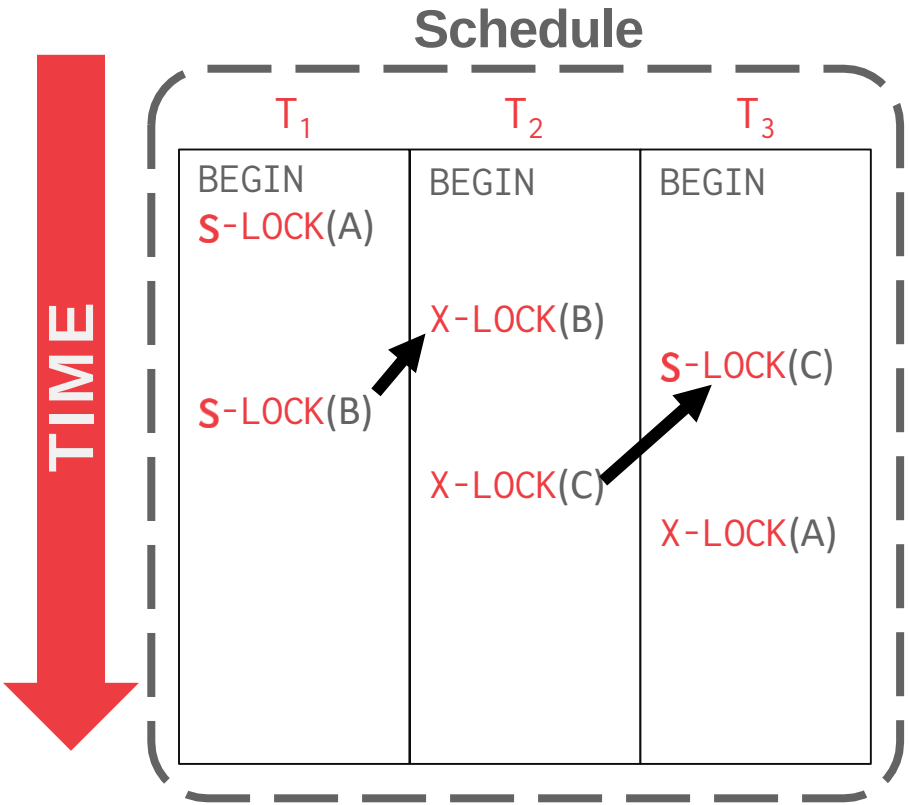
Deadlock Detection

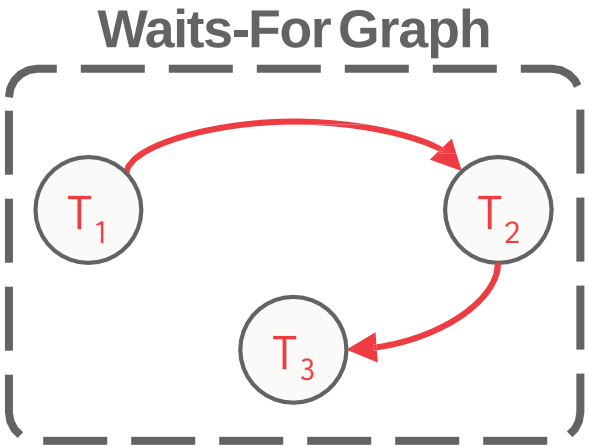




Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

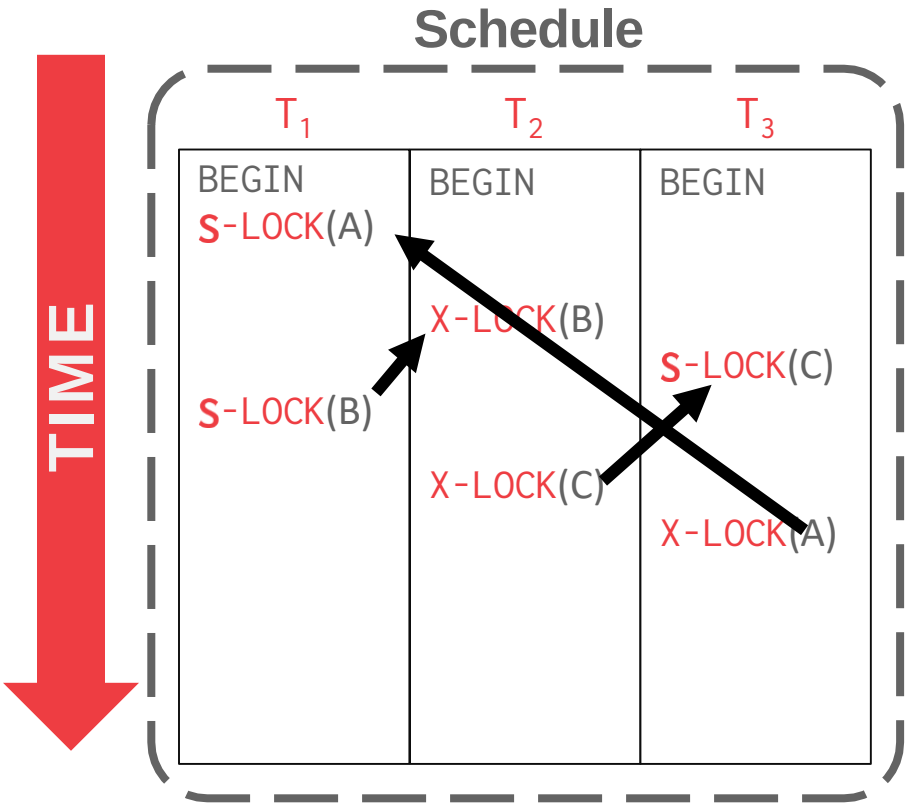
Deadlock Detection

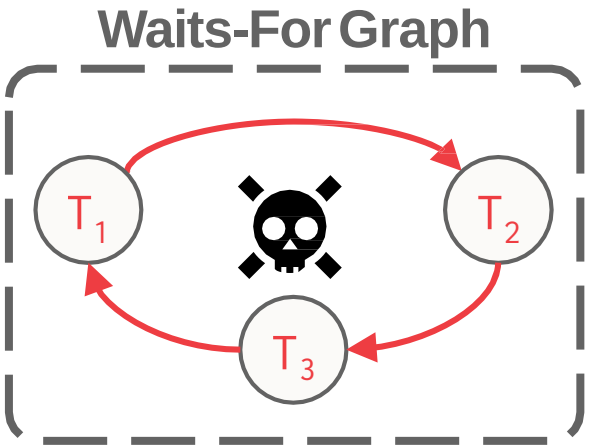




Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Detection





Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Handling

- When the DBMS detects a deadlock, it will select a "victim" txn to rollback to break the cycle.

- The victim txn will either restart or abort(more common) depending on how it was invoked.

- There is a trade-off between the frequency of checking for deadlocks and how long txns have to wait before deadlocks are broken.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Handling: Victim Selection

Selecting the proper victim depends on a lot of different variables….

→ By age (lowest timestamp)
→ By progress (least/most queries executed)
→ By the # of items already locked
→ By the # of txns that we have to rollback with it

We also should consider the # of times a txn has been restarted in the past to prevent starvation.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Handling: Rollback Length

After selecting a victim txn to abort, the DBMS can also decide on how far to rollback the txn's changes.

Approach #1: Completely

Approach #2: Minimally

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Prevention

When a txn tries to acquire a lock that is held by another txn, the DBMS kills one of them to prevent a deadlock.

This approach does <u>not</u> require a **waits-for** graph or detection algorithm.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Conditions for Deadlock

- **Mutual exclusion**: processes require exclusive control of its resources (not sharing)
- **Hold and wait**: process may wait for a resource while holding others.
- **No preemption**: process will not give up a resource until it is finished with it. Also, **processes are irreversible**: unable to reset to an earlier state where resources not held.
- **Circular wait**: each process in the chain holds a resource requested by another

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Conservative (static) 2PL

- Requires a transaction to lock all the items it accesses before the transaction begins
  - Predeclare read-set and write-set
- Deadlock-free protocol

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Dealing with Deadlock and Starvation (cont'd.)

- Deadlock prevention protocols
  - Every transaction locks all items it needs in advance
  - Ordering all items in the database
    - Transaction that needs several items will lock them in that order
  - Both approaches impractical
- Protocols based on a timestamp
  - Wait-die
  - Wound-wait

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Prevention

Assign priorities based on timestamps:

→ Older Timestamp = Higher Priority (e.g., $T_1 > T_2$)

Wait-Die ("Old Waits for Young")

→ If *requesting txn* has higher priority than *holding txn*, then *requesting txn* waits for *holding txn*.

→ Otherwise *requesting txn* aborts.

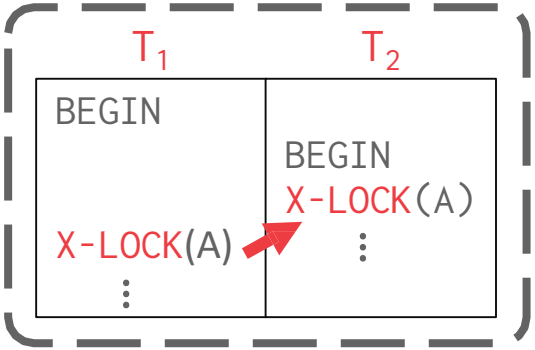
Wound-Wait ("Young Waits for Old")

→ If *requesting txn* has higher priority than *holding txn*, then *holding txn* aborts and releases lock.

→ Otherwise *requesting txn* waits.

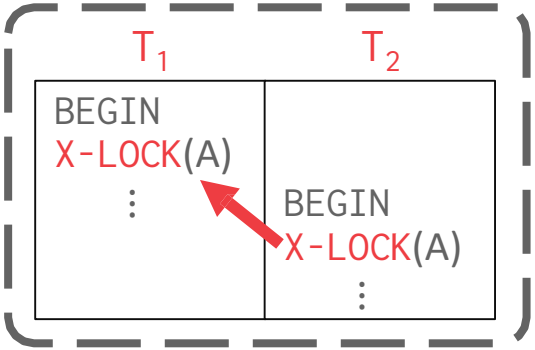
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Prevention



Wait-Die: T_1 waits

Wound-Wait: T_2 aborts



Wait-Die: T_2 aborts

Wound-Wait: T_2 waits

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Prevention

Why do these schemes guarantee no deadlocks ?

Only one "type" of direction allowed when waiting for a lock.

When a txn restarts, what is it (new) priority?

Its original timestamp. Why?

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Deadlock Prevention Without Timestamps

- ***No waiting***: If T needs X and X is already locked, immediately abort T, restart later.
  - May cause lots of restarting
- ***Cautious waiting***: Suppose T_i needs X, and X is already locked by T_j.

  If T_j is waiting for anything, then abort T_j

  else abort T_i.
- ***Timeout***: If T waits longer than some fixed time, abort and restart T.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Starvation and Livelock

- Another snake in the Garden of Eden
- A T might be aborted and restarted indefinitely: *Starvation*
- A T might wait indefinitely, not for the same other T (deadlock), but for different reasons at different times: *Livelock*
- Footnote: Some people use "starvation" for either situation.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Outline

- Lock Granularity and Database Lock Hierarchy
- Multi-granularity Locks

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Observation

- All of these examples have a one-to-one mapping from database objects to locks.

- If a txn wants to update one billion tuples, then it has to acquire one billion locks.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Lock Granularity

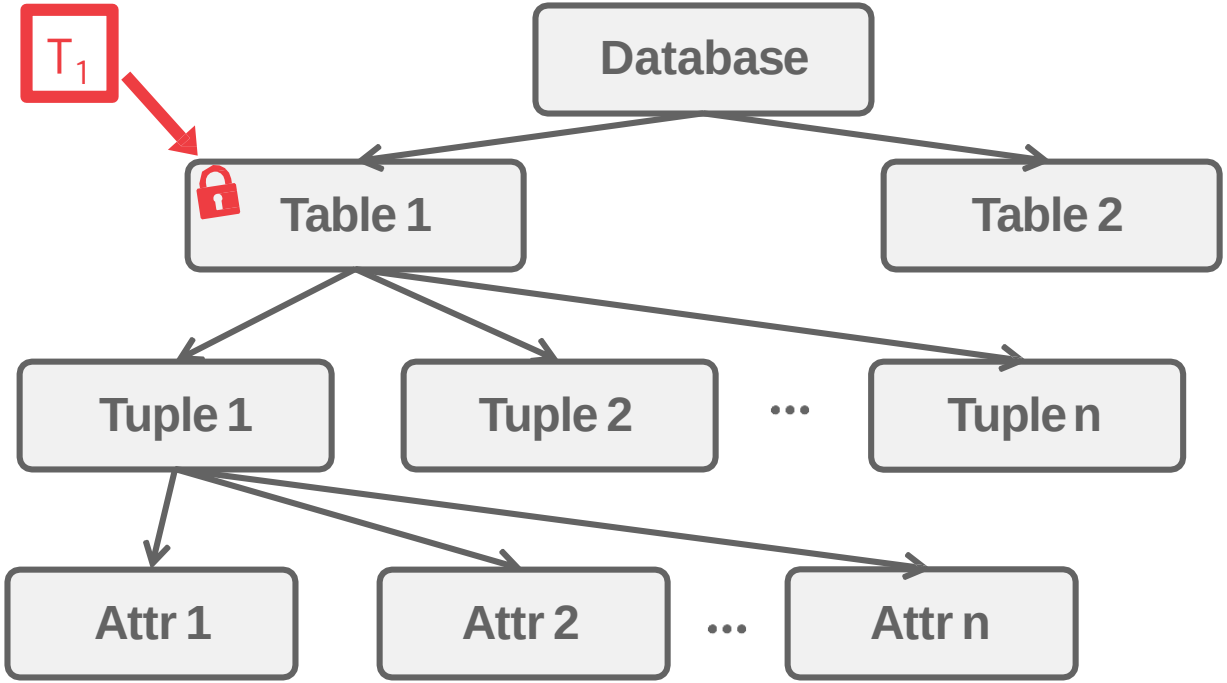
When we say that a txn acquires a "lock", what does that actually mean?

→ On an Attribute? Tuple? Page? Table?

Ideally, each txn should obtain fewest number of locks that is needed…

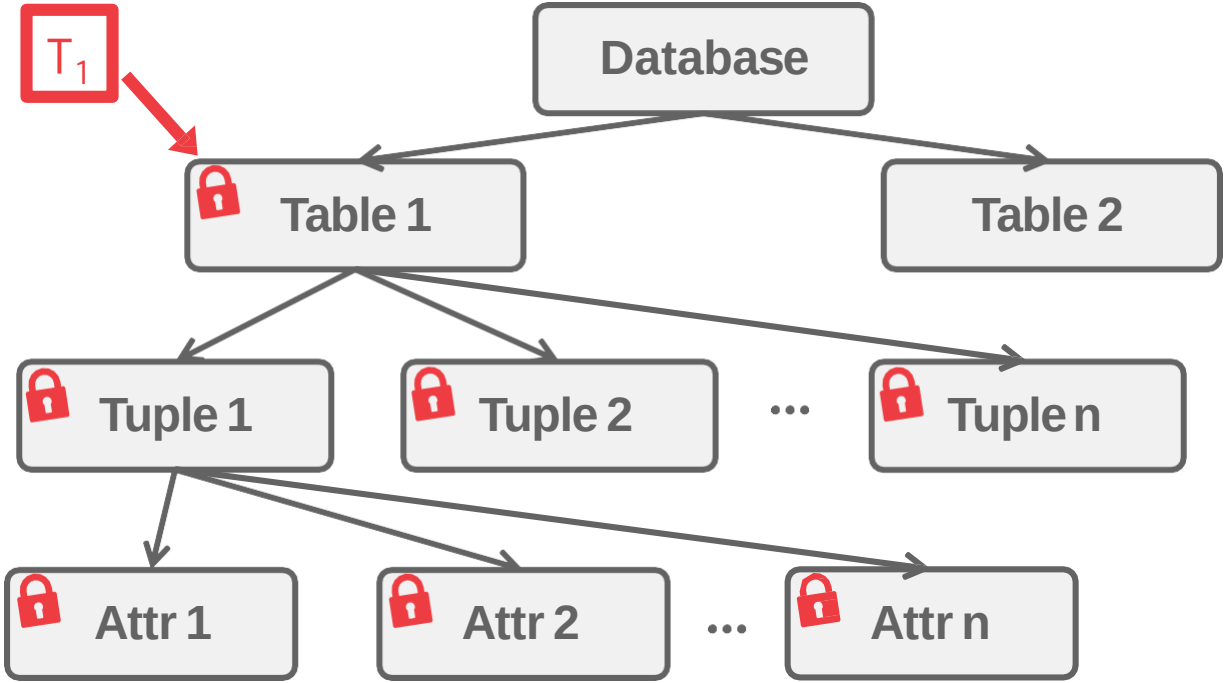
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Database Lock Hierarchy



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

DATABASE LOCK HIERARCHY



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE

T_1 – Get the balance of Andy's shady off-shore bank account.

T_2 – Increase Matt's bank account balance by 1%

What locks should these txns obtain?.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE

T_1 – Get the balance of Andy's shady off-shore bank account.

T_2 – Increase Matt's bank account balance by 1%

. **What locks should these txns obtain?.**

Multiple:

→ **Exclusive** + **Shared** for leafs of lock tree.

→ Special **Intention** locks for higher levels.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Intention Locks

An **intention lock** allows a higher level node to be locked in **shared** or **exclusive** mode without having to check all descendent nodes.

If a node is in an intention mode, then explicit locking is being done at a lower level in the tree.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Intention Locks

Intention-Shared (S)

→ Indicates explicit locking at a lower level with shared locks.

Intention-Exclusive (IX)

→ Indicates locking at lower level with exclusive or shared locks.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

INTENTION LOCKS

Shared+Intention-Exclusive (SIX)

→ The subtree rooted by that node is locked explicitly in **shared** mode and explicit locking is being done at a lower level with **exclusive-mode** locks.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

COMPATIBILITY MATRIX

T_2 Wants

T_1 Holds

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
SIX	✓	×	×	×	×
X	×	×	×	×	×

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Locking Protocol

Each txn obtains appropriate lock at highest level of the database hierarchy.

To get S or IS lock on a node, the txn must hold at least IS on parent node.

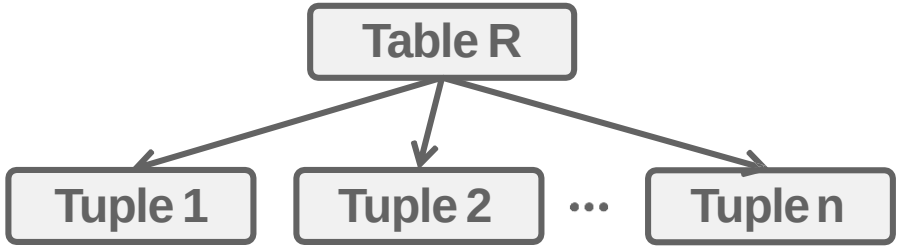
To get X, IX, or SIX on a node, must hold at least IXon parent node.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example Two-level Hierarchy

Read Andy's record in **R**.

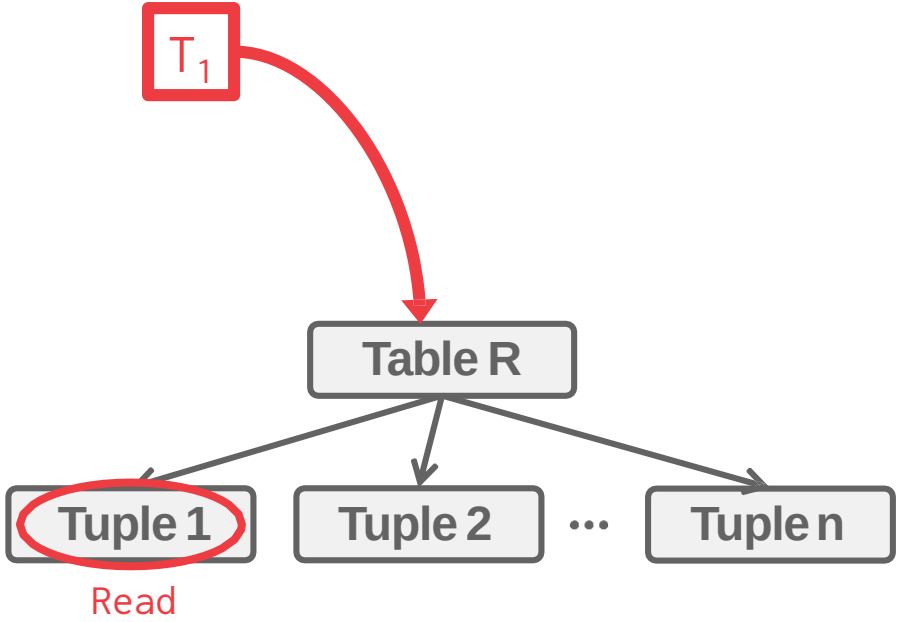
T_1



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example Two-level Hierarchy

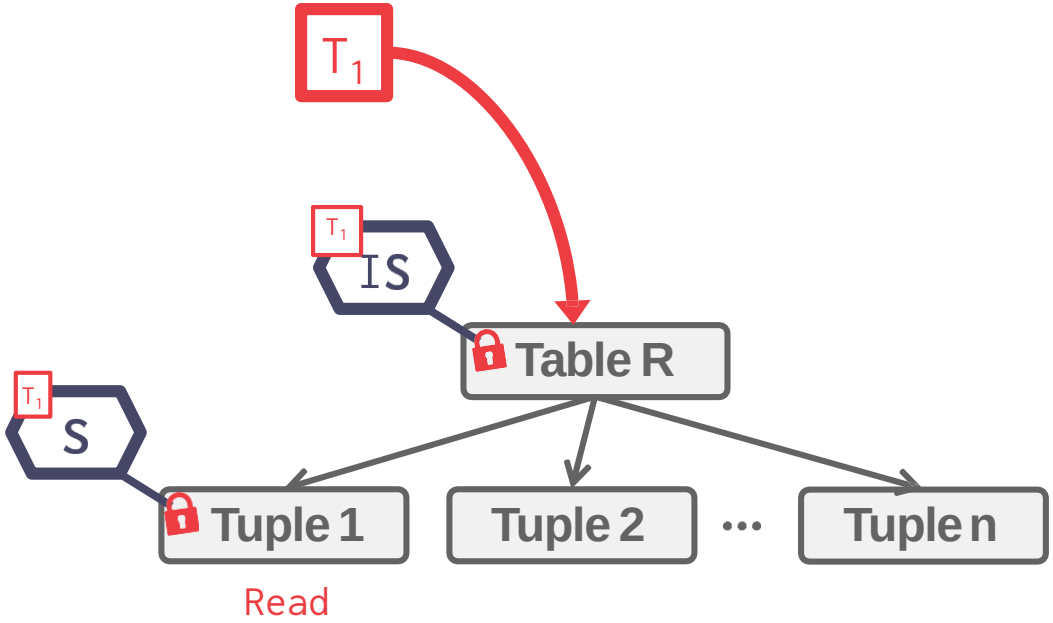
Read Andy's record in **R**.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example Two-level Hierarchy

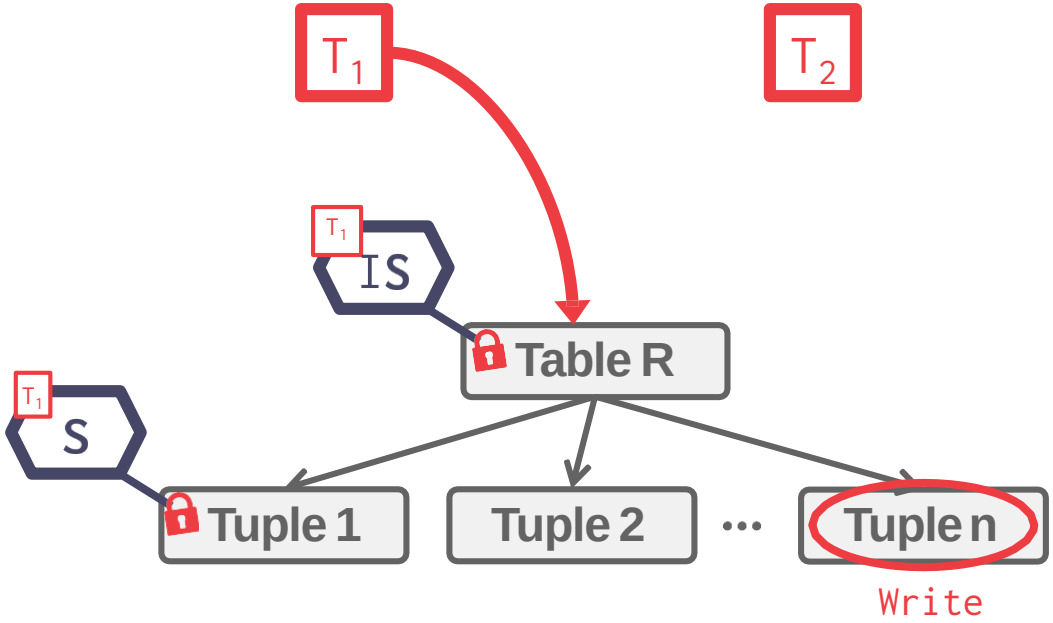
Read Andy's record in **R**.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example Two-level Hierarchy

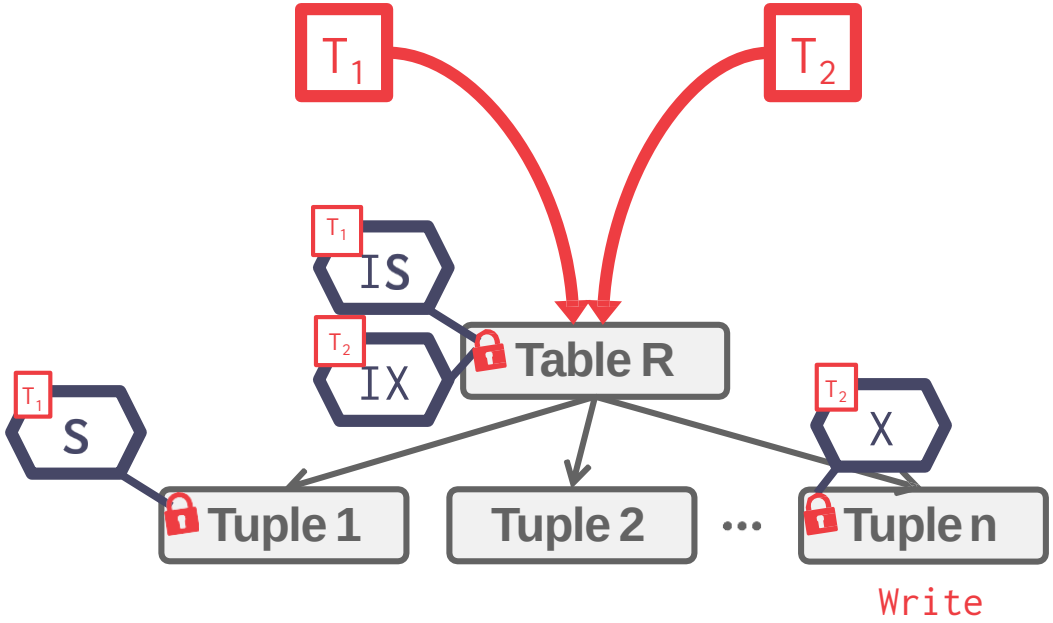
Update Matt's record in **R**.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example Two-level Hierarchy

Update Matt's record in **R**.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

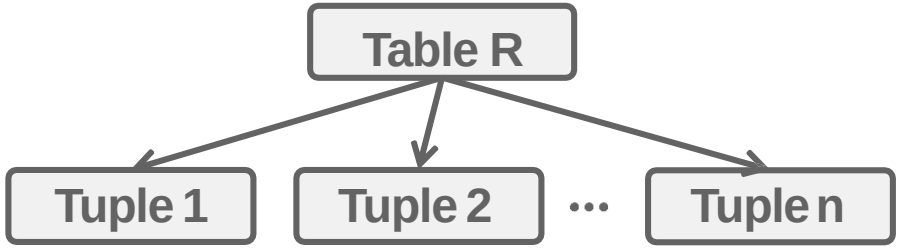
Example-Threesome

Assume three txns execute at same time:

→ T_1 – Scan R and update a few tuples.

→ T_2 – Read a single tuple in R.

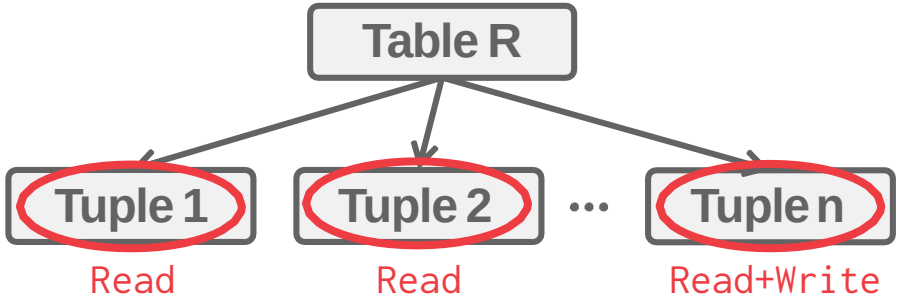
→ T_3 – Scan all tuples in R.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example-Threesome

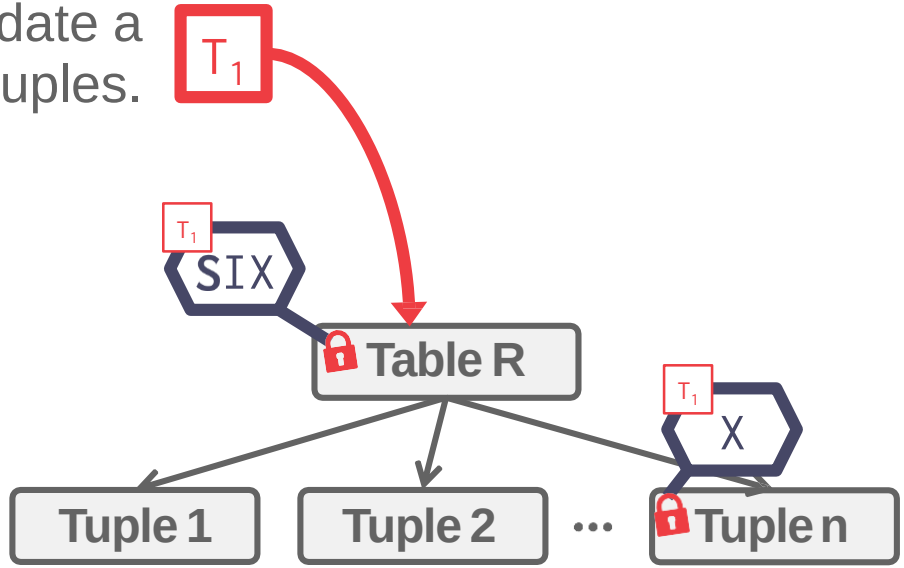
Scan R and update a few tuples. T_1



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

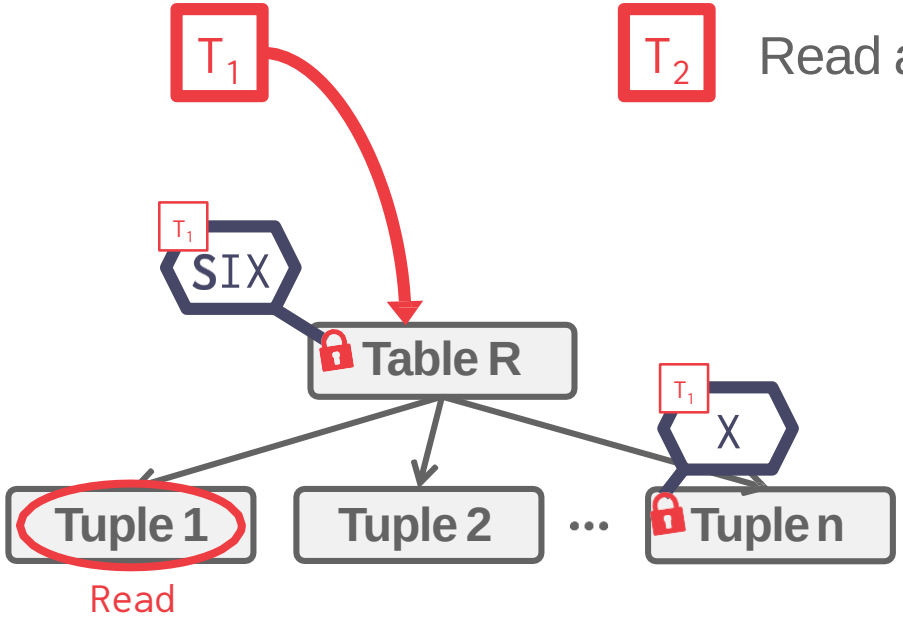
Example-Threesome

Scan R and update a few tuples.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

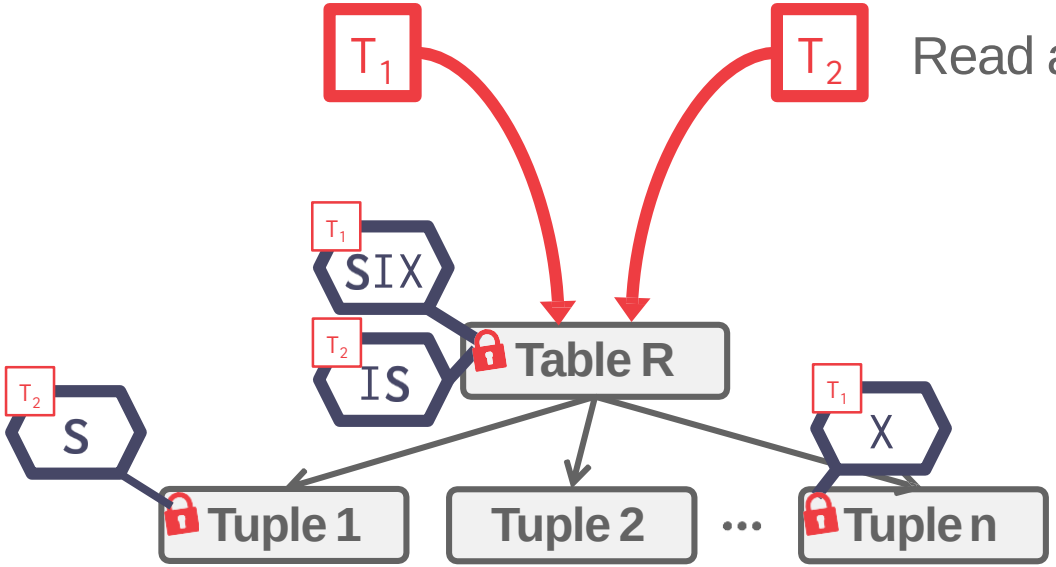
Example-Threesome



Read a single tuple in R.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example-Threesome

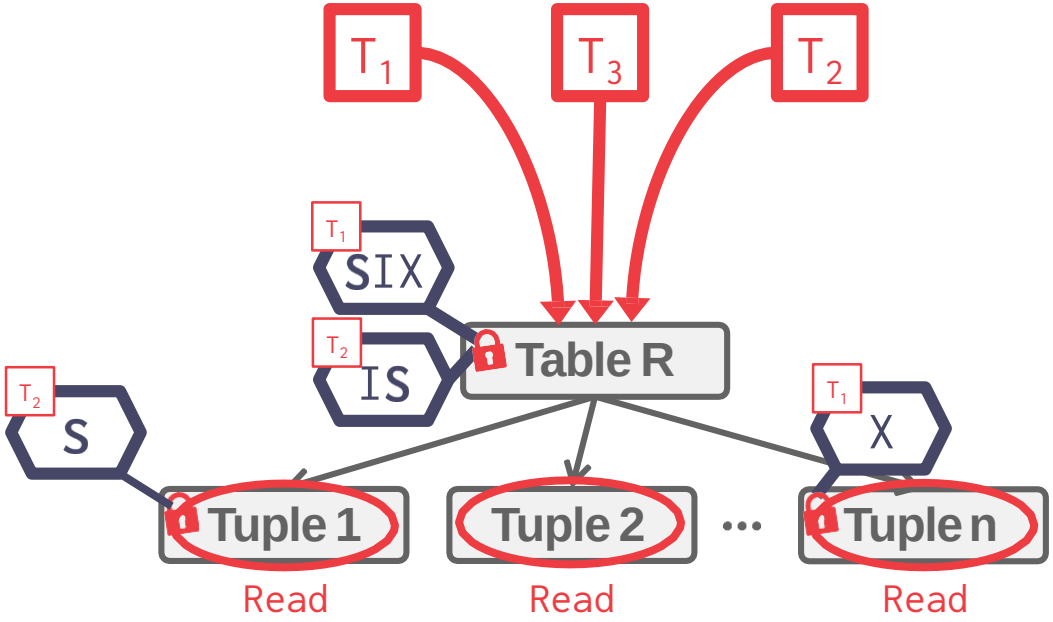


Read a single tuple in R.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example-Threesome

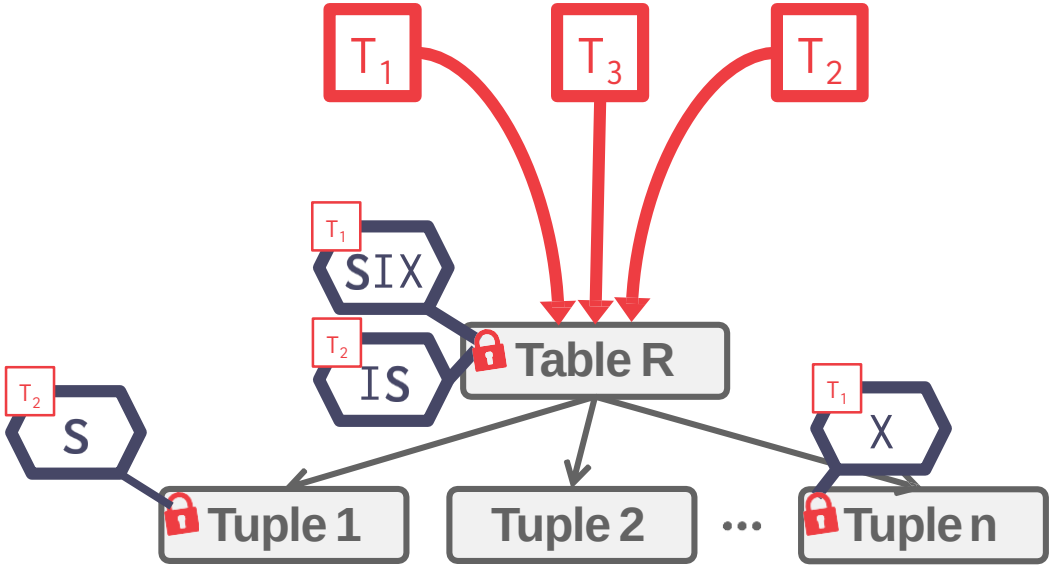
Scan all tuples in R.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example-Threesome

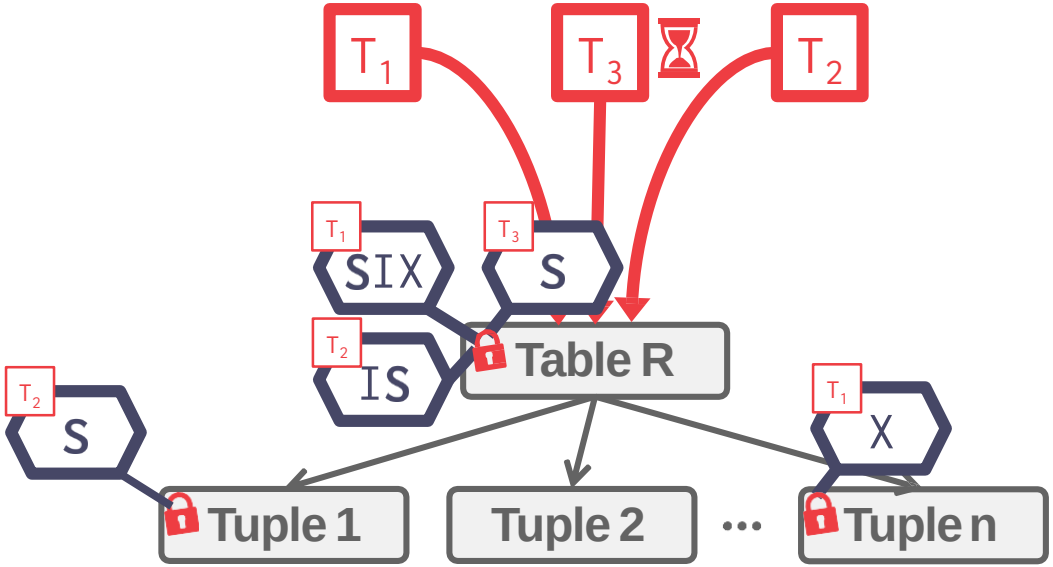
Scan all tuples in R.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Example-Threesome

Scan all tuples in R.



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Multiple Lock Granularities

Hierarchical locks are useful in practice as each txn only needs a few locks.

Intention locks help improve concurrency:

→ **Intention-Shared**(IS): Intent to get s lock(s) at finer granularity

→**Intention-Exclusive**(IX): Intent to get x lock(s) at finer granularity

→ **Shared-Intension-Exclusive**(SIX): Like S and IX at the same time

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Lock Escalation

- Lock escalation is the process of converting many fine-grain locks to fewer coarse-grain locks, which reduces memory overhead at the cost of decreasing concurrency.

- This also reduces the number of requests that the locker manger has to process

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Locking in Practice

You typically don't set locks manually in txns.

Sometimes you will need to provide the DBMS with hints to help it to improve concurrency.

Explicit locks are also useful when doing major changes to the database.

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

LOCK TABLE

Explicitly locks a table.

Not part of the SQL standard.

→ Postgres/DB2/Oracle Modes: SHARE, EXCLUSIVE

→ MySQL Modes: READ, WRITE

PostgreSQL

ORACLE

IBM DB2

```
LOCK TABLE <table> IN <mode> MODE;
```

Microsoft SQL Server

```
SELECT 1 FROM <table> WITH (TABLOCK, <mode>);
```

MySQL

```
LOCK TABLE <table> <mode>;
```

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

SELECT...FOR SHARE

Perform a select and then sets an share lock on the matching tuples.

→ Postgres: FOR SHARE
→ MySQL: LOCK IN SHARE MODE

```
SELECT * FROM <table>
WHERE <qualification> LOCK IN SHARE MODE;
```

```
SELECT * FROM <table>
WHERE <qualification> FOR SHARE;
```

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

SELECT...FOR UPDATE

Perform a select and then sets an exclusive lock on the matching tuples.

```
SELECT * FROM <table>
WHERE <qualification> FOR UPDATE;
```

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

Conclusion

2PL is used in almost DBMS.

Automatically generates correct interleaving:

→ Locks + protocol (2PL, SS2PL ...)

→ Deadlock detection + handling

→ Deadlockprevention

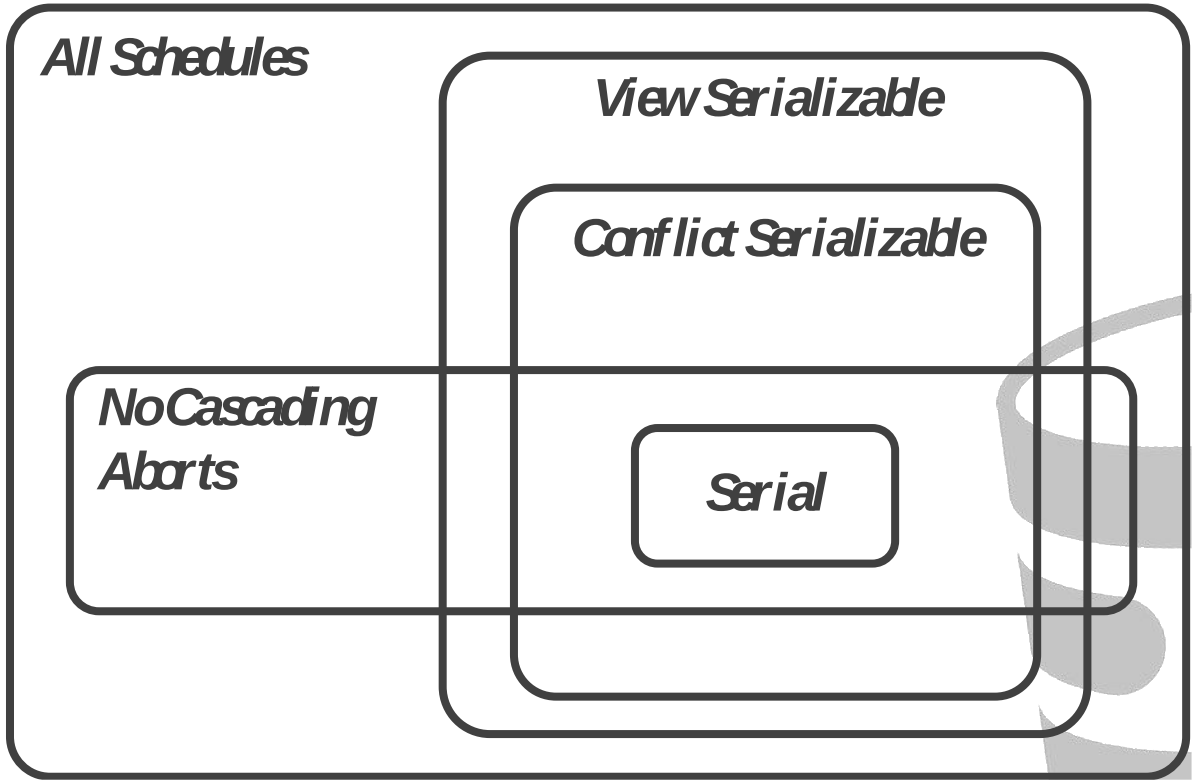
Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

NEXT CLASS

Timestamp Ordering Concurrency Control

Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

UNIVERSE OF SCHEDULES



Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe